Today’s Outline

- Neural Network design and it’s training

DALL-E 2

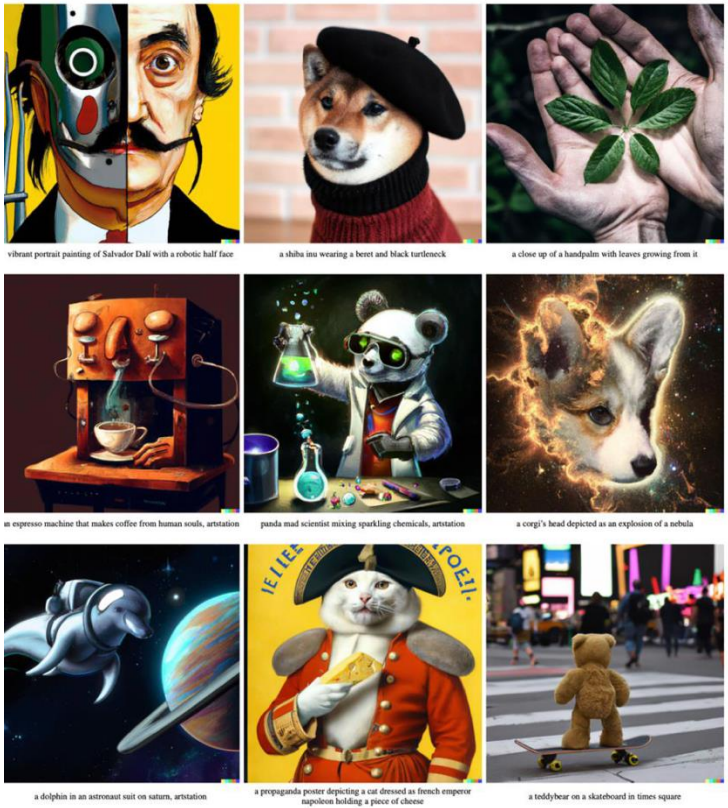
"Teddy bears working on new AI research on the moon in the 1980s."

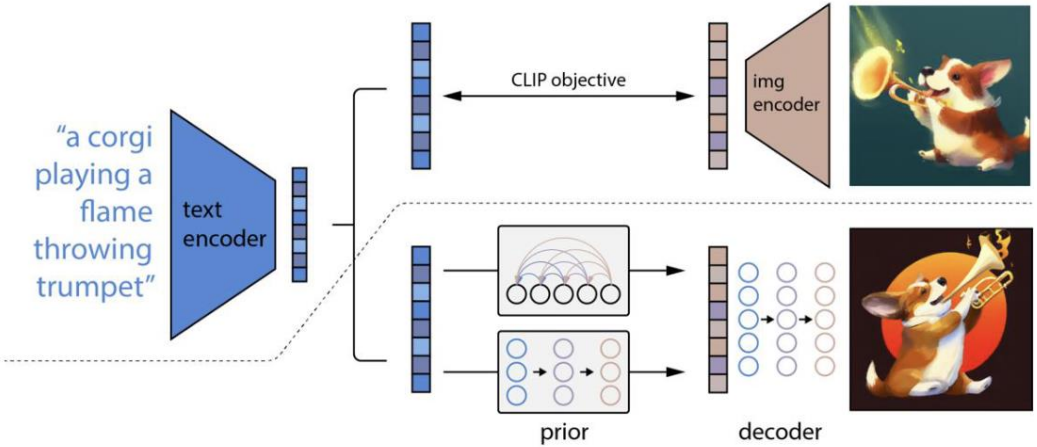
"Rabbits attending a college seminar on human anatomy."

"A wise cat meditating in the Himalayas searching for enlightenment."

Text to image generation

DALL-E 2





Ramesh et al., Hierarchical Text-Conditional Image Generation with CLIP Latents, 2022.

DALL-E 3

In a fantastical setting, ahighly detailed furry humanoid skunk with piercing eyes confidently poses in a medium shot, wearing an animal hidejacket. The artist hasmaster fully rendered the character in digital art, capturing the intricate details of fur and clothing texture.

DALL-E 3

An illustration from a graphic novel. A bustling city street under the shine of a full moon. The sidewalks bustling with pedestrians enjoyingthe nightlife. At the corner stall, a young woman with fiery red hair, dressed in a signature velvet cloak, is haggling with the grumpy oldvendor. The grumpy vendor, a tall, sophisticated man wearing a sharp suit, who sports a noteworthymustache is animatedly conversing on his steampunk telephone.

GPT-4

User What is unusual about this image?

Source: Barnorama

GPT-4 The unusual thing about this image is that a man is ironing clothes on an ironing board attached to the roof of a moving taxi.

User Can you explain this meme?

GPT-4 This meme is a joke that combines two unrelated things: pictures of the earth from space and chicken nuggets.
The text of the meme suggests that the image below is a beautiful picture of the earth from space. However, the image is actually of chicken nuggets arranged to vaguely resemble a map of the world.
The humor in this meme comes from the unexpected juxtaposition of the text and the image. The text sets up an expectation of a majestic image of the earth, but the image is actually something mundane and silly.

Multimodal understanding

Segment Anything Model (SAM)

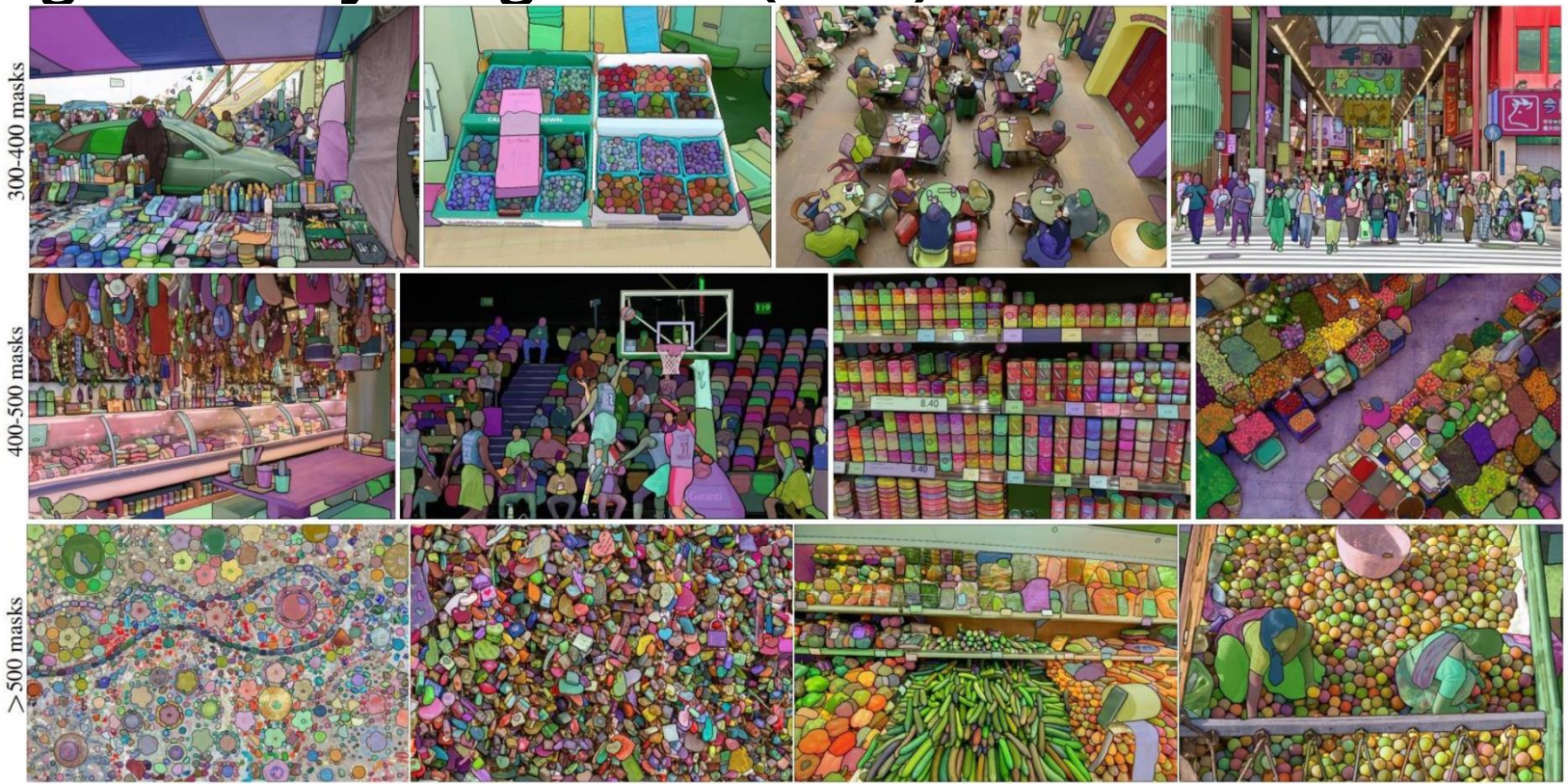


Image understanding and segmentation

Neural networks: the original linear classifier

(Before) Linear score function: $f = Wx$

Neural networks: the original linear classifier

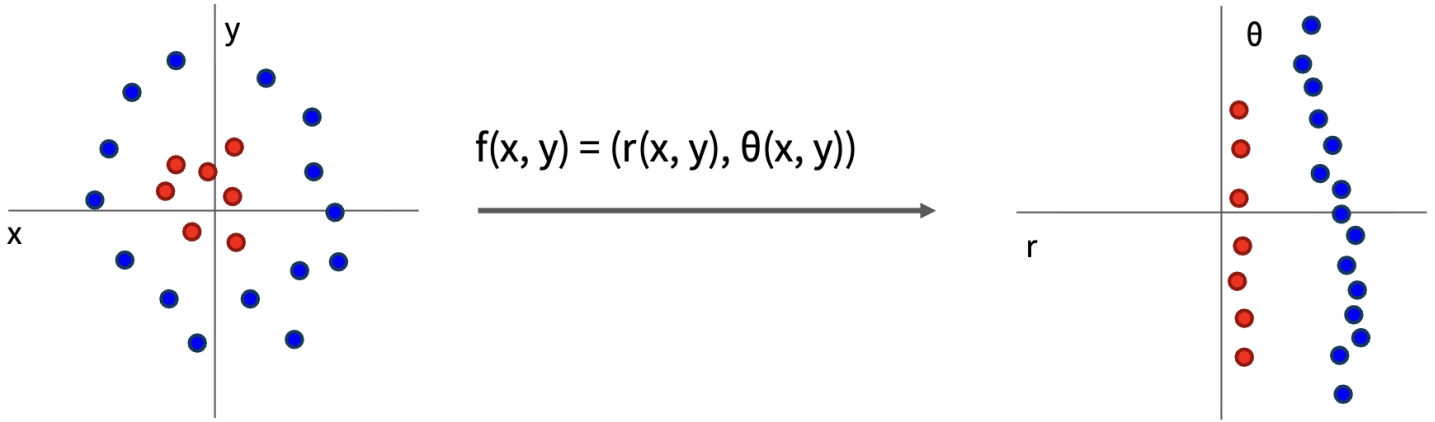
(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$

(In practice we will usually add a learnable bias at each layer as well)

Why do we want non-linearity?



Cannot separate red and blue points with linear classifier

After applying feature transform, points can be separated by linear classifier

Non-linearity enables complex functionality.

Neural networks: the original linear classifier

(Before) Linear score function: $f = Wx$

(Now) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$

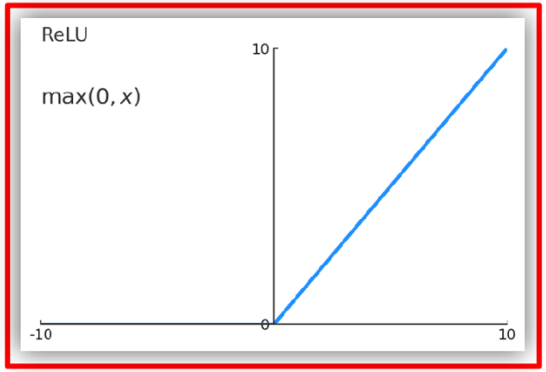
$x \in \mathbb{R}^D, W_1 \in \mathbb{R}^{H \times D}, W_2 \in \mathbb{R}^{C \times H}$

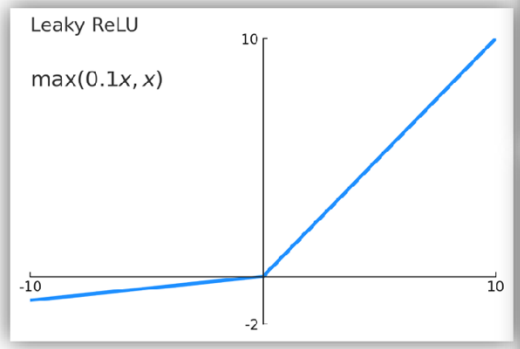
(In practice we will usually add a learnable bias at each layer as well)

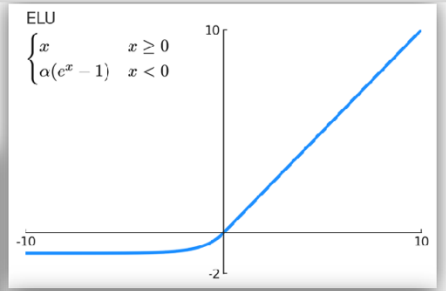
Choices for Activation

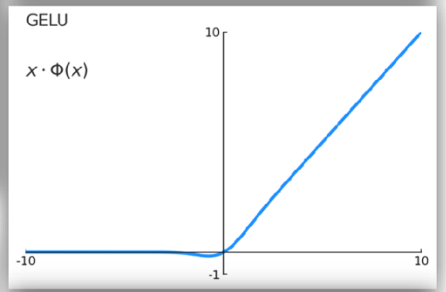
Activation functions

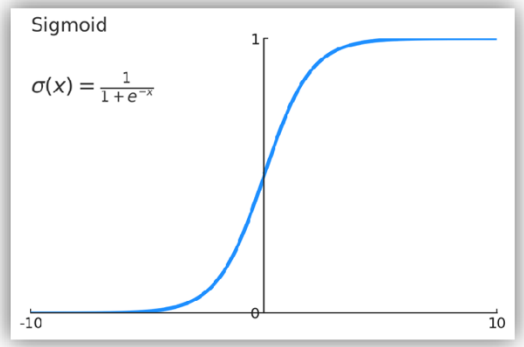
ReLU is a good default choice for most problems

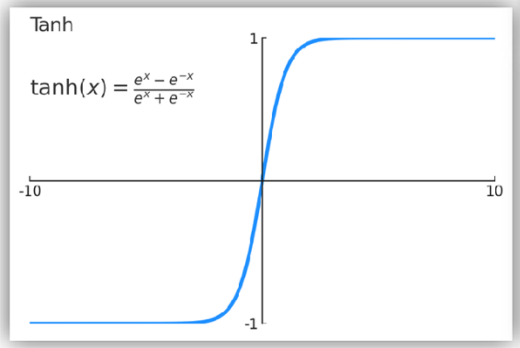


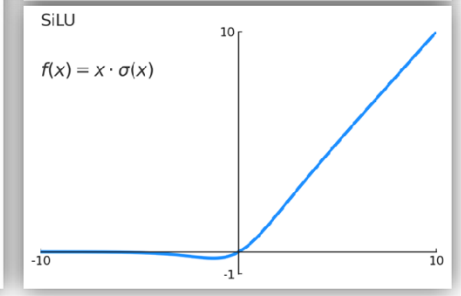




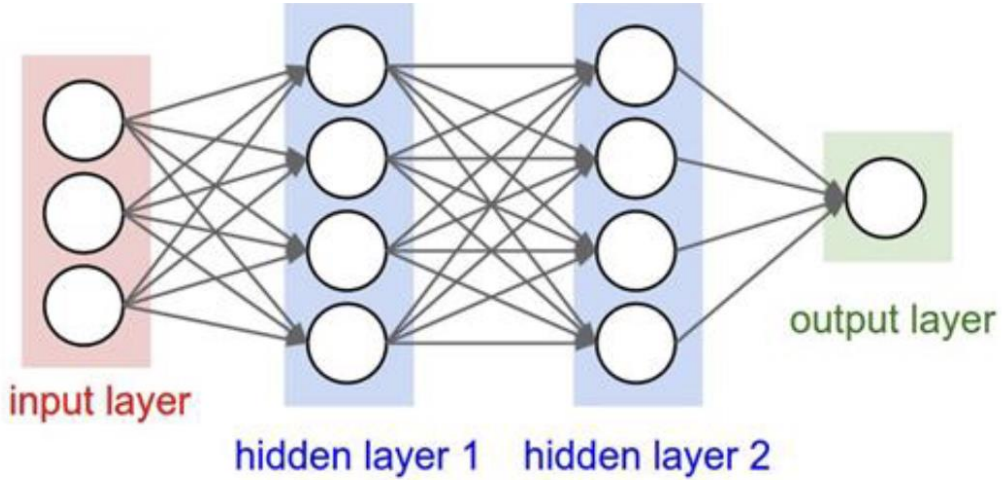








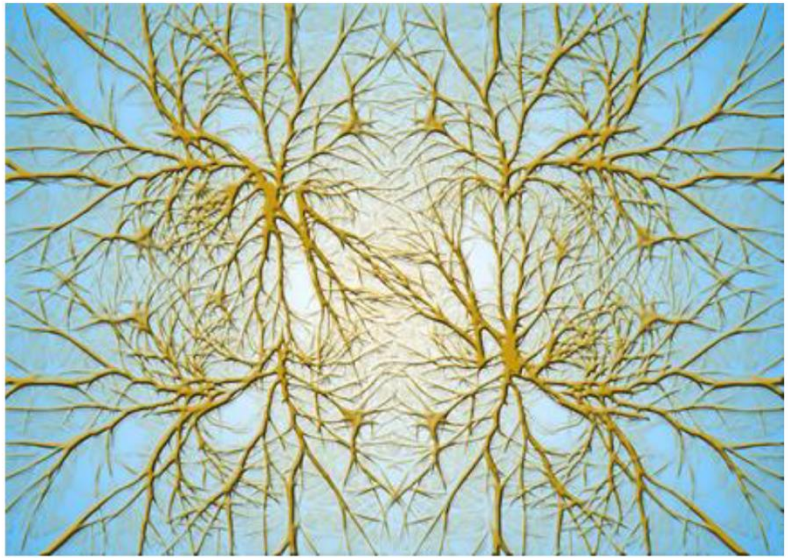
Example computation of a neural network



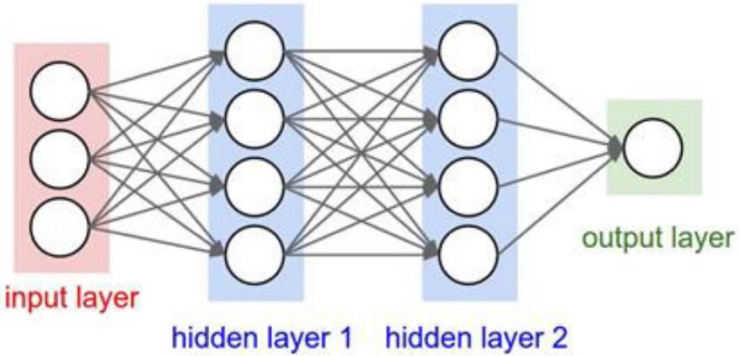

```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

Comparison: Biological and Artificial Neurons

Biological Neurons:
Complex connectivity patterns



Neurons in a neural network:
Organized into regular layers for computational efficiency



Problem: How to compute gradients?

$$s = f(x; W_1, W_2) = W_2 \max(0, W_1 x)$$ Nonlinear score function

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$ Hinge Loss on predictions

$$R(W) = \sum_k W_k^2$$ Regularization

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i + \lambda R(W_1) + \lambda R(W_2)$$ Total loss: data loss + regularization

If we can compute $\frac{\partial L}{\partial W_1}, \frac{\partial L}{\partial W_2}$ then we can learn W_1 and W_2

Backpropagation in Neural Network

Backpropagation: a simple example

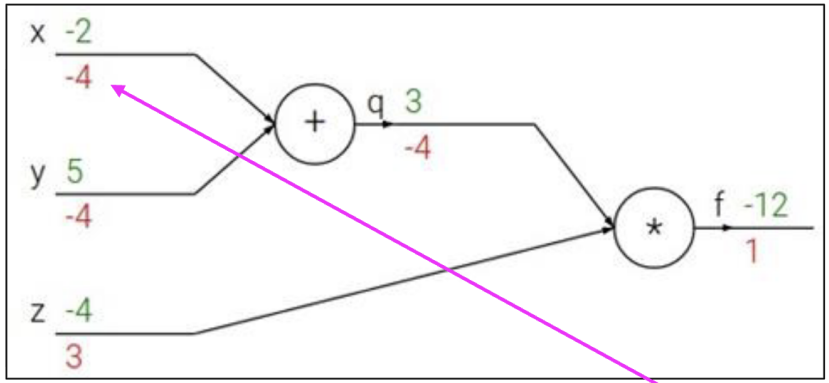
$$f(x, y, z) = (x + y)z$$

e.g. x = -2, y = 5, z = -4

$$q = x + y \qquad \frac{\partial q}{\partial x} = 1, \frac{\partial q}{\partial y} = 1$$

$$f = qz \qquad \frac{\partial f}{\partial q} = z, \frac{\partial f}{\partial z} = q$$

Want: $\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z}$



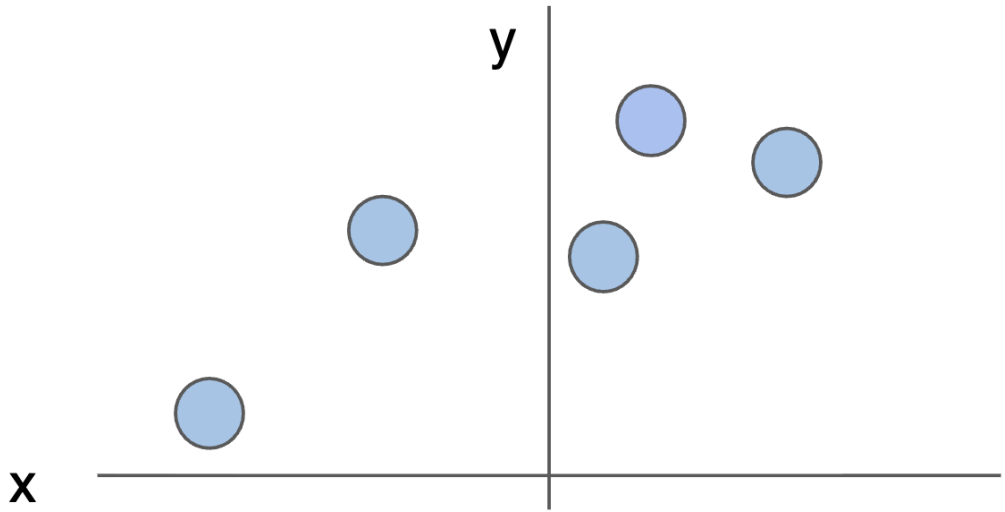
Chain rule:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q}\frac{\partial q}{\partial x}$$

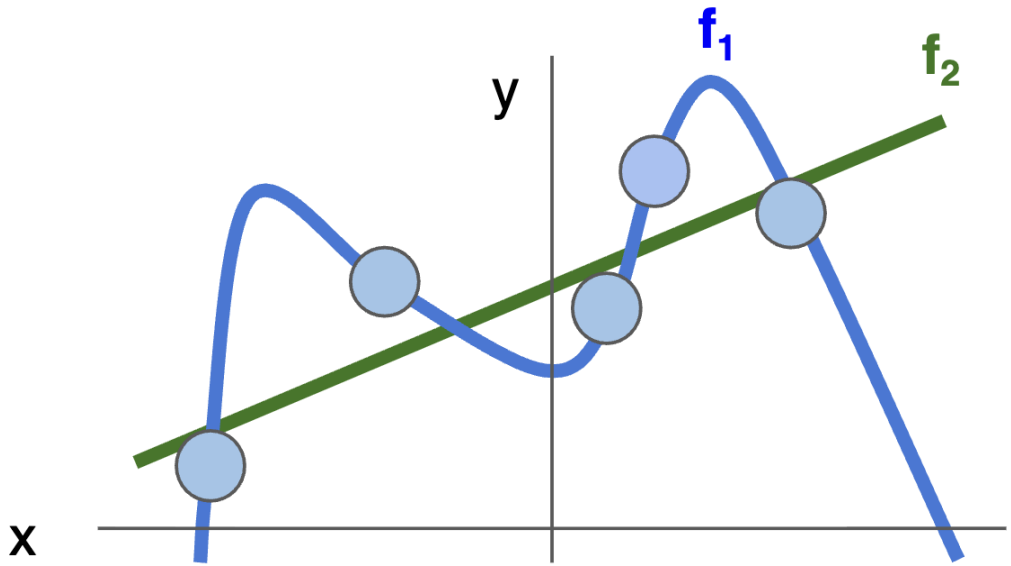
Upstream gradient

Local gradient

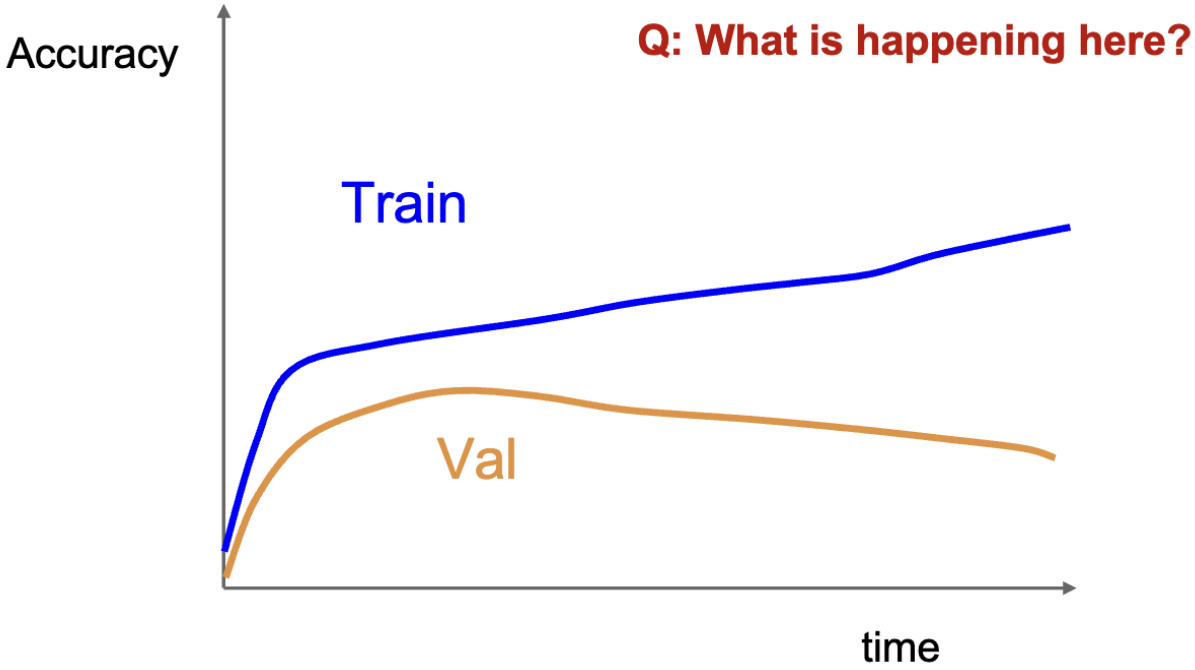
Regularization intuition: toy example training data



Regularization intuition: Prefer Simpler Models



Regularization intuition: Prefer Simpler Models



Regularization

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

Occam's Razor: Among multiple competing hypotheses, the simplest is the best, William of Ockham 1285-1347

Regularization

$$L(W) = \underbrace{\frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i)}_{\text{Data loss}} + \underbrace{\lambda R(W)}_{\text{Regularization}}$$

Data loss: Model predictions should match training data

Regularization: Prevent the model from doing *too* well on training data

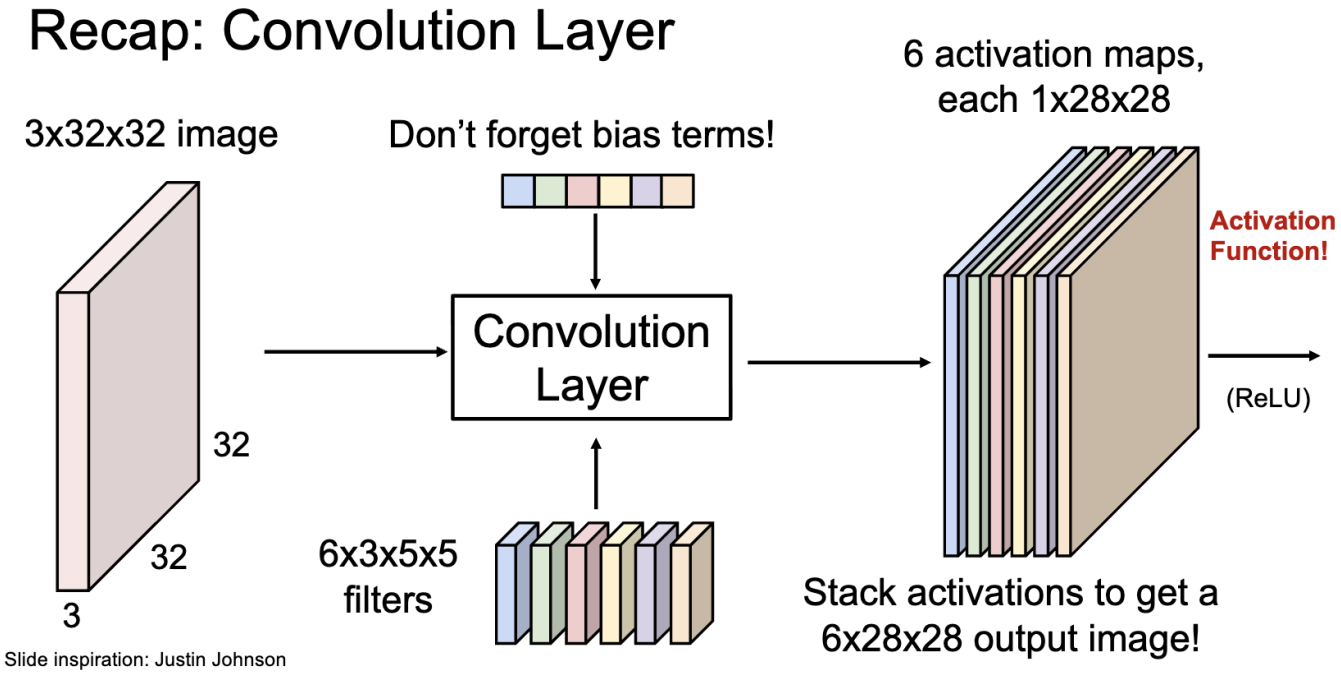
Simple examples

L2 regularization: $R(W) = \sum_k \sum_l W_{k,l}^2$

L1 regularization: $R(W) = \sum_k \sum_l |W_{k,l}|$

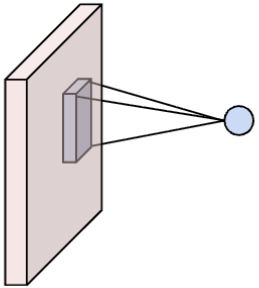
Elastic net (L1 + L2): $R(W) = \sum_k \sum_l \beta W_{k,l}^2 + |W_{k,l}|$

How to build CNNs?

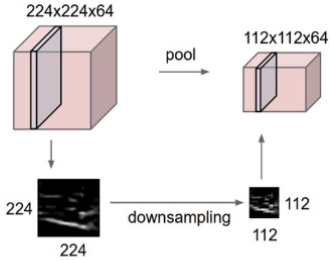


Components of CNNs

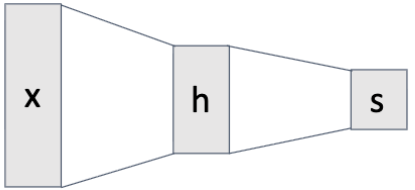
Convolution Layers



Pooling Layers



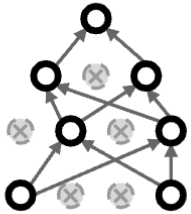
Fully-Connected Layers



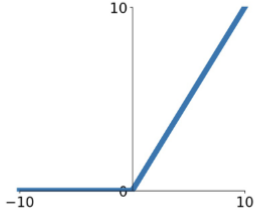
Normalization Layers

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \varepsilon}}$$

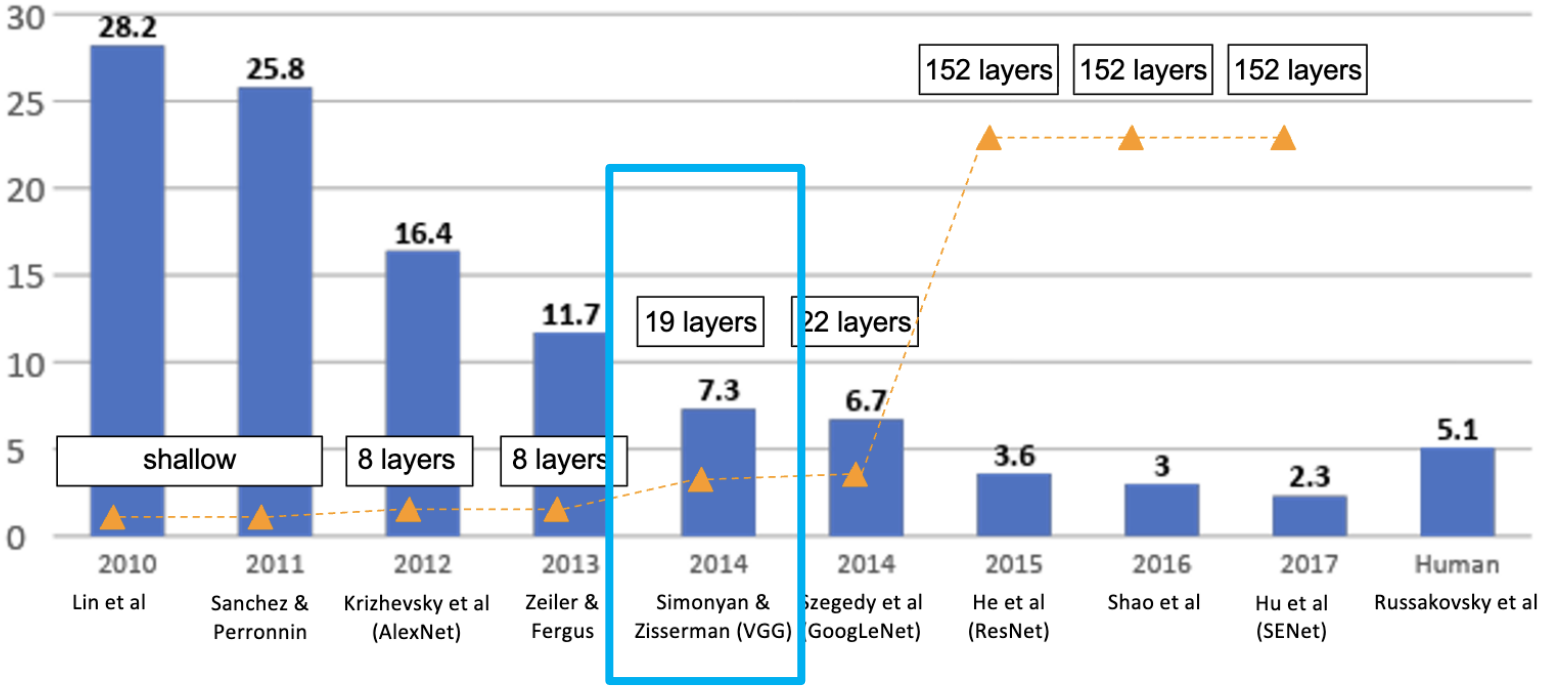
Dropout (sometimes)



Activation Functions



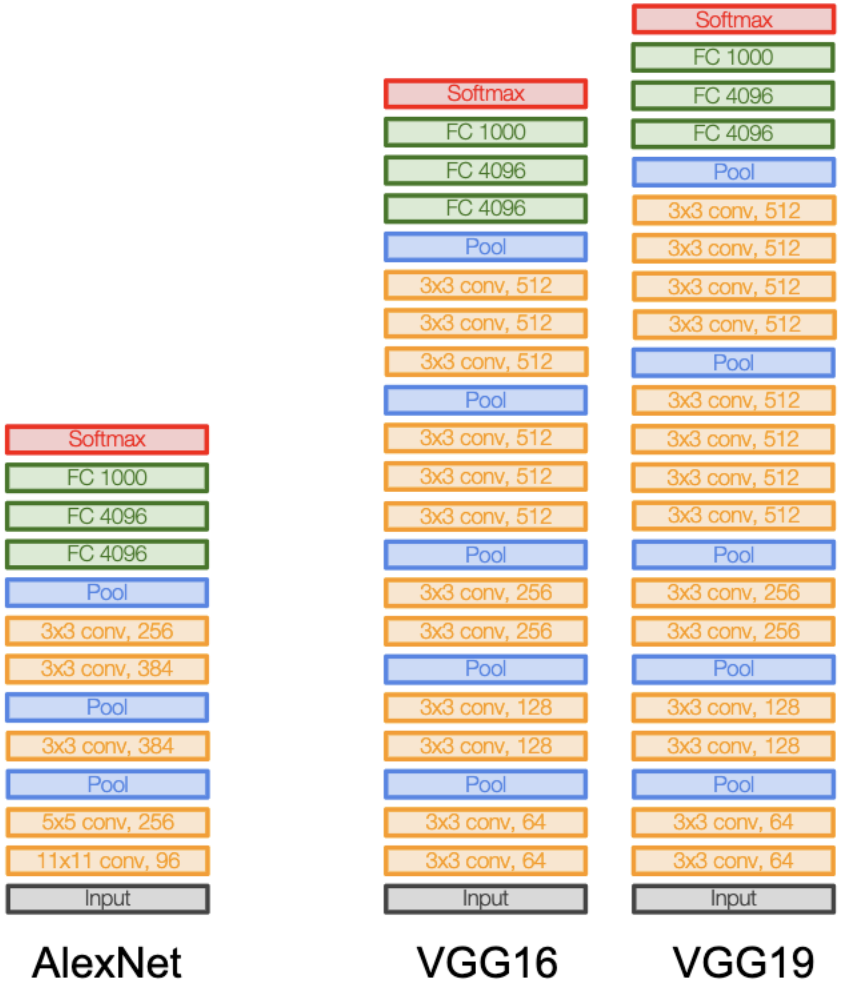
ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



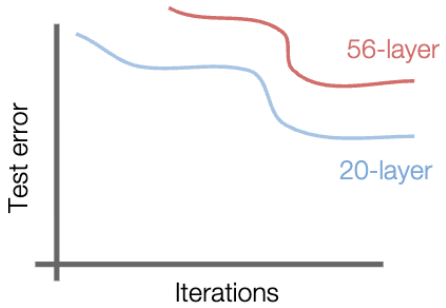
Case Study: VGGNet

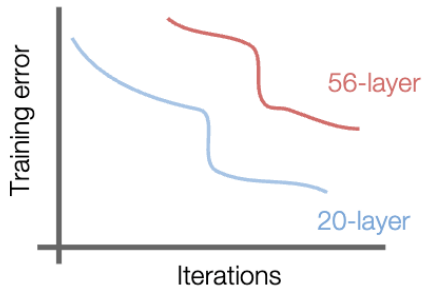
Small filters, Deeper networks

- 8 layers (AlexNet)-> 16 - 19 layers (VGG16Net)
- Only 3x3 CONV stride 1, pad 1and 2x2 MAX POOL stride 2
- 11.7% top 5 error in ILSVRC'13(ZFNet)
- -> 7.3% top 5 error in ILSVRC'14



Case Study: ResNet





- Fact: Deep models have more representation power(more parameters) than shallower models.

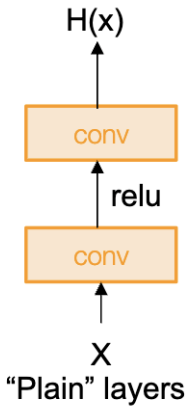
- Hypothesis: the problem is an optimization problem,deeper models are harder to optimize

What should the deeper model learn to be at leastas good as the shallower model?

What happens when we continue stacking deeper layers on a “plain” convolutional neural network?

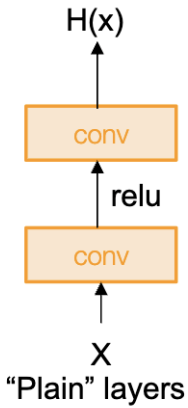
Case Study: ResNet

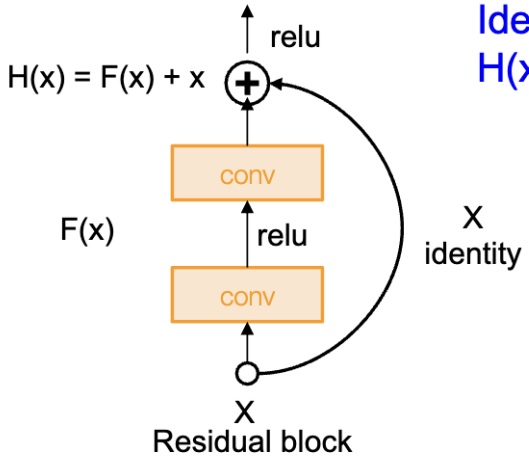
Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



Case Study: ResNet

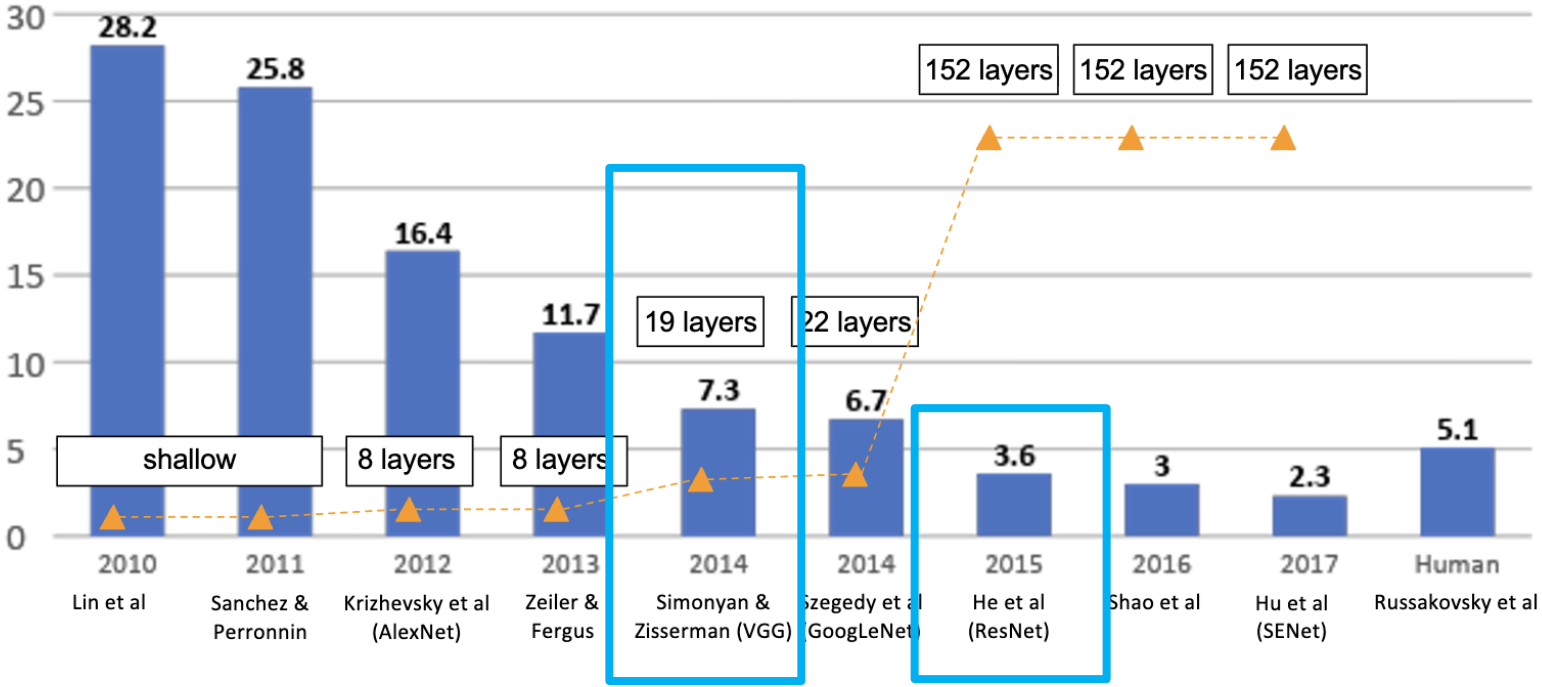
Solution: Use network layers to fit a residual mapping instead of directly trying to fit a desired underlying mapping



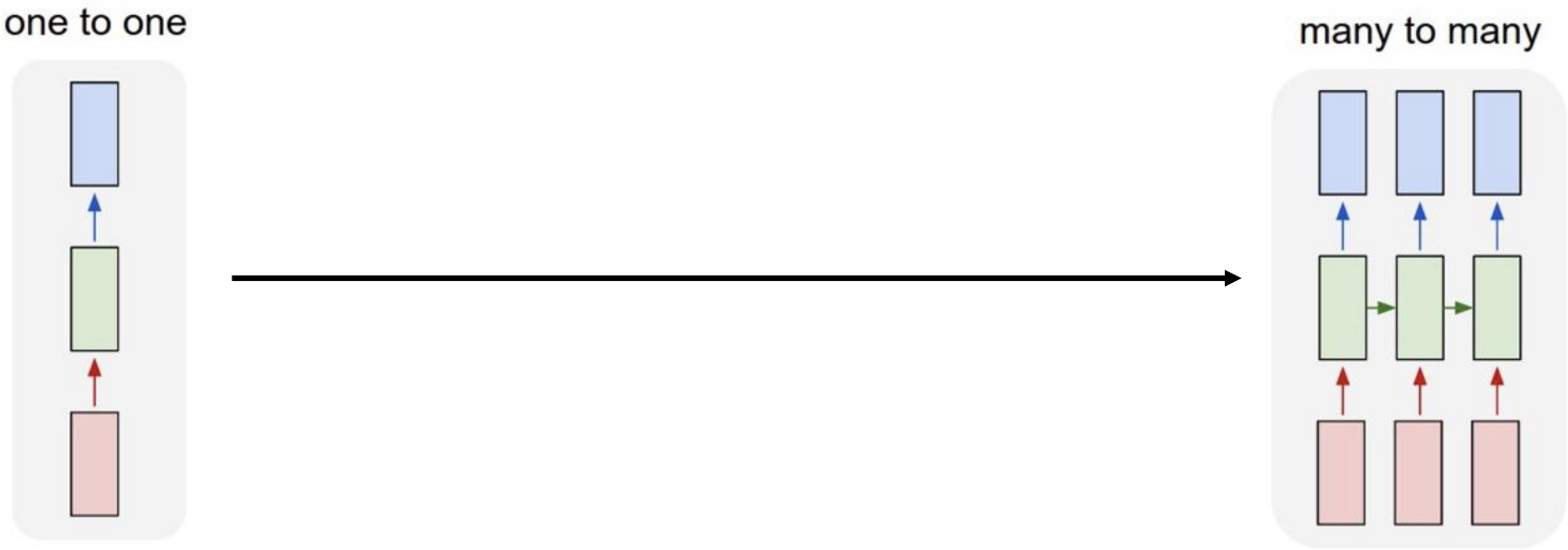


Identity mapping:
H(x) = x if F(x) = 0

ImageNet Large Scale Visual Recognition Challenge (ILSVRC) winners



From One-to-One to Many-to-Many



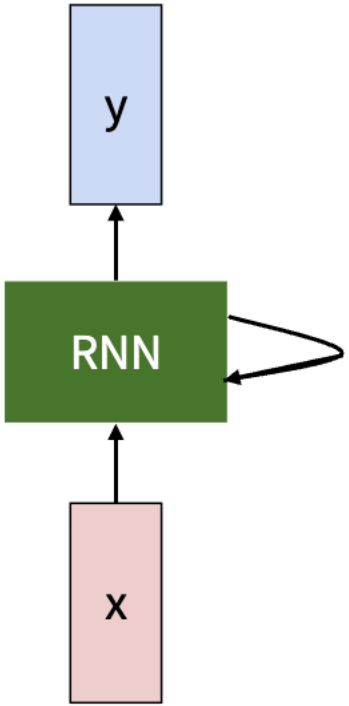
Vanilla Neural Networks

(e.g., image recognition)

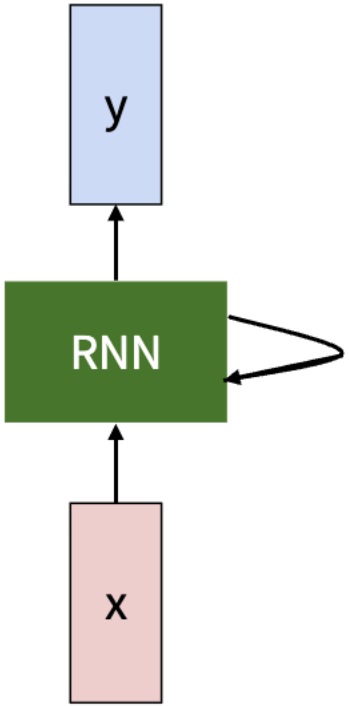
Sequential Neural Networks

(e.g., Video classification on frame level)

Recurrent Neural Network

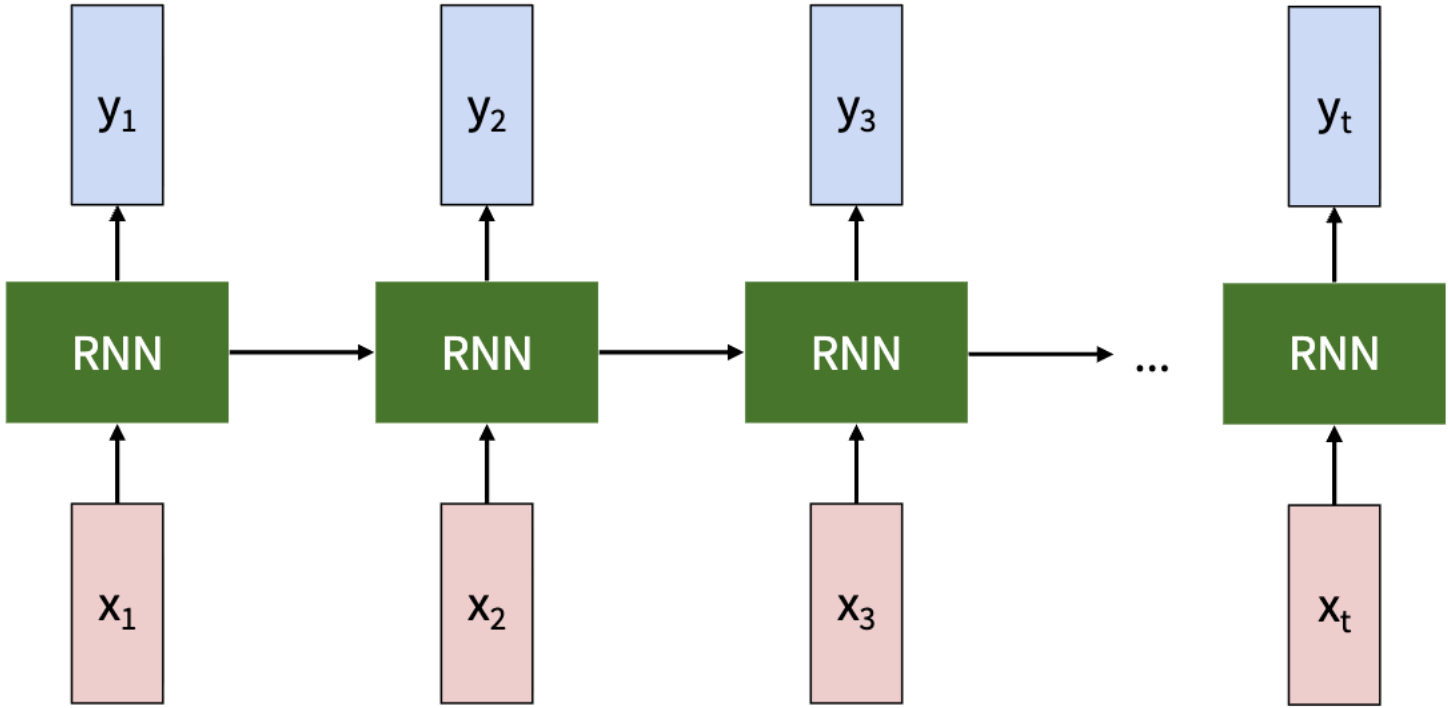


Recurrent Neural Network



Key idea: RNNs have an “internal state” that is updated as a sequence is processed

Unrolled RNN



Recurrent Neural Network

We can process a sequence of vectors x by applying a recurrence formula at every time step:

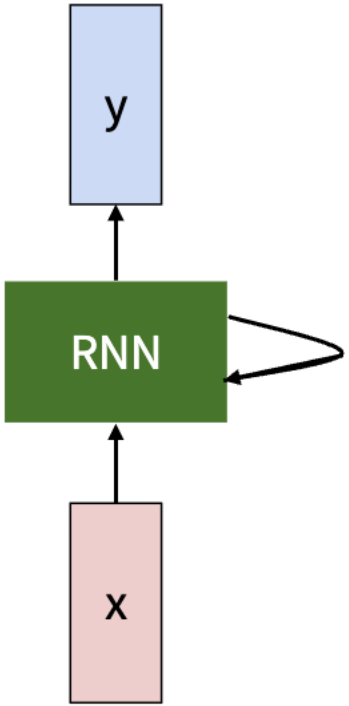
$$h_t = f_W(h_{t-1}, x_t)$$

new state

some function with parameters W

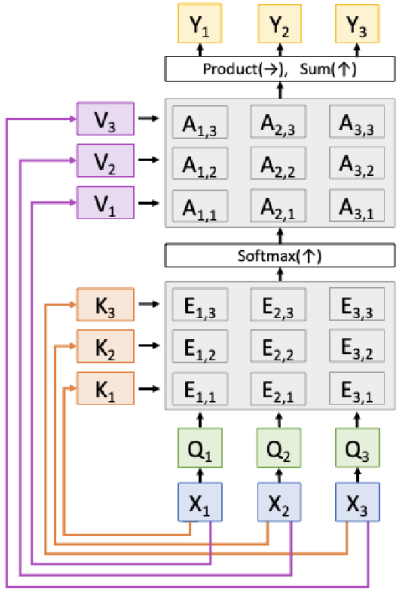
old state

input vector at some time step

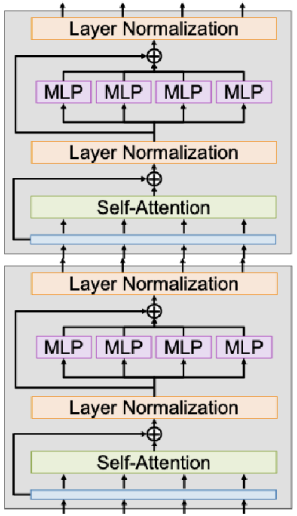


Attention + Transformers

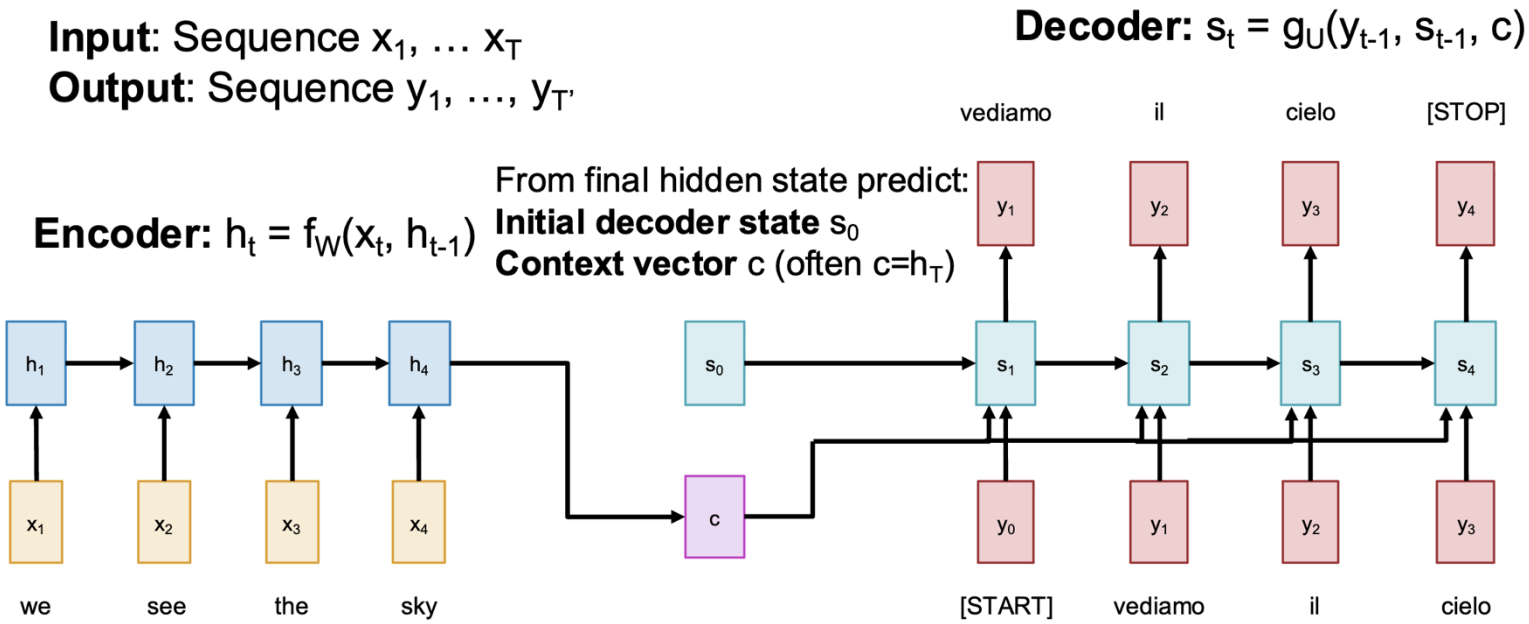
Attention: A new primitive that operates on sets of vectors



Transformer: A neural network architecture that uses attention everywhere



Motivation of Attention: Sequence to Sequence with RNNs



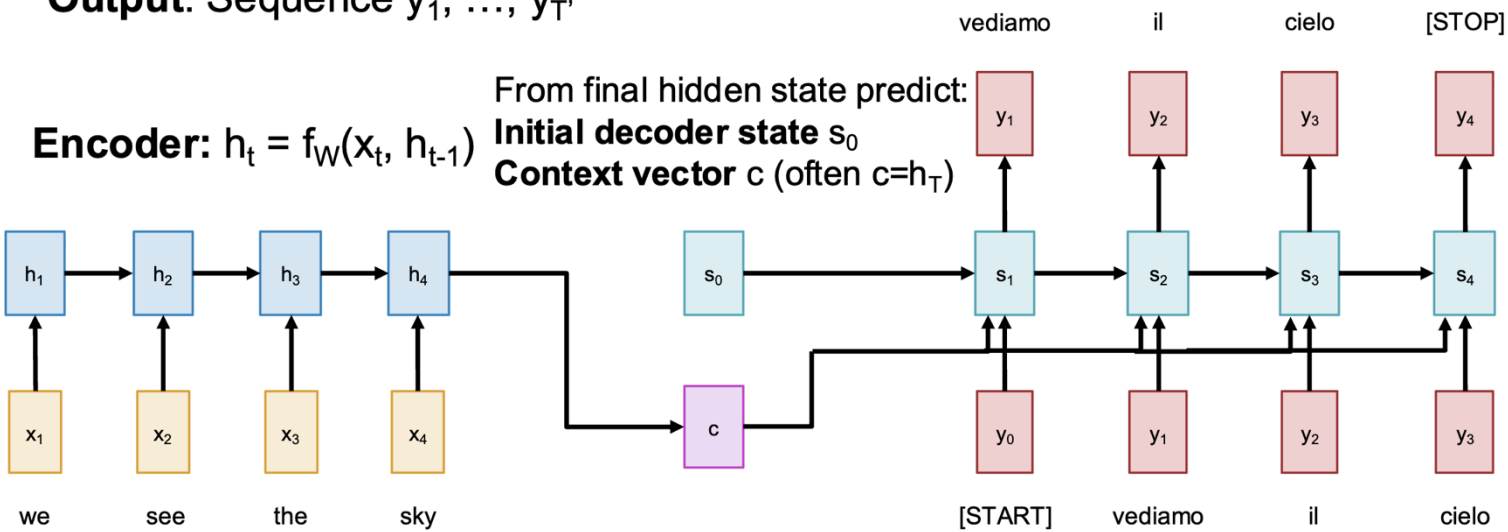
Motivation of Attention: Sequence to Sequence with RNNs

Input: Sequence $x_1, \ldots x_T$
Output: Sequence $y_1, \ldots, y_{T'}$

Encoder: $h_t = f_W(x_t, h_{t-1})$

Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

From final hidden state predict:
Initial decoder state s_0
Context vector c (often $c=h_T$)

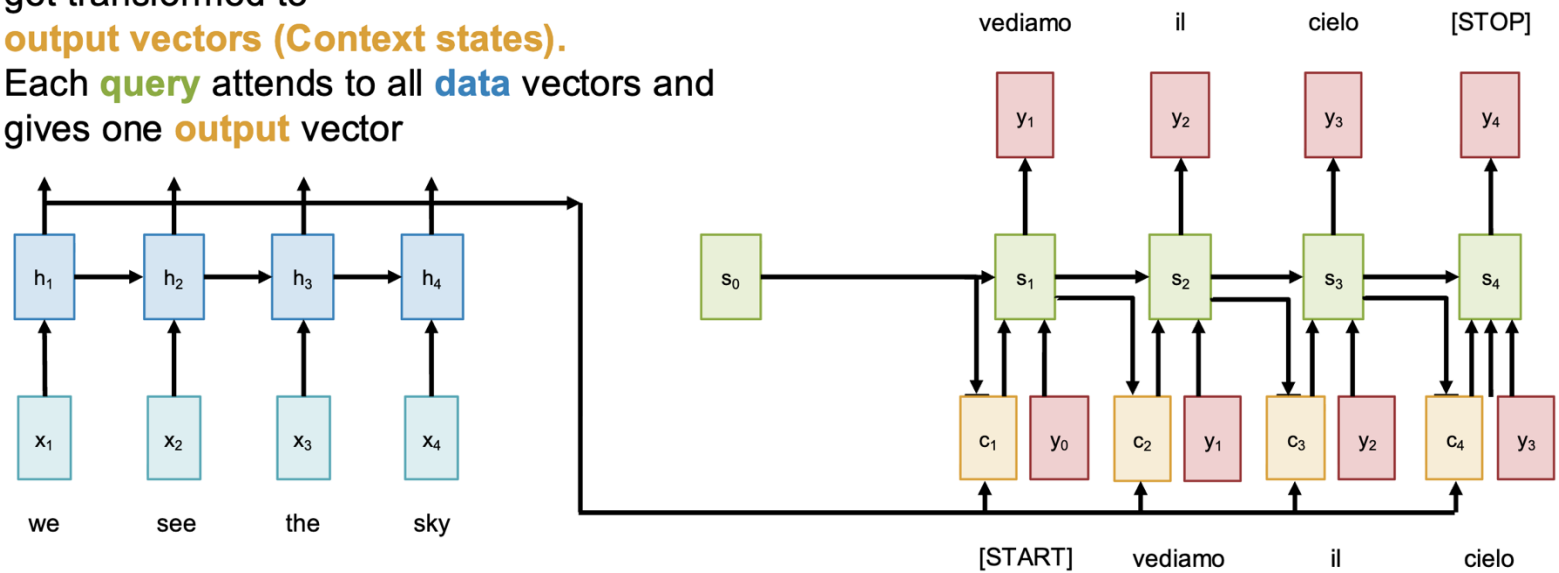


Solution: Look back at the whole input sequence on each step of the output

Sequence to Sequence with RNNs and Attention

Query vectors (decoder RNN states) and **data vectors (encoder RNN states)** get transformed to **output vectors (Context states).** Each **query** attends to all **data** vectors and gives one **output** vector

There's a general operator hiding here:

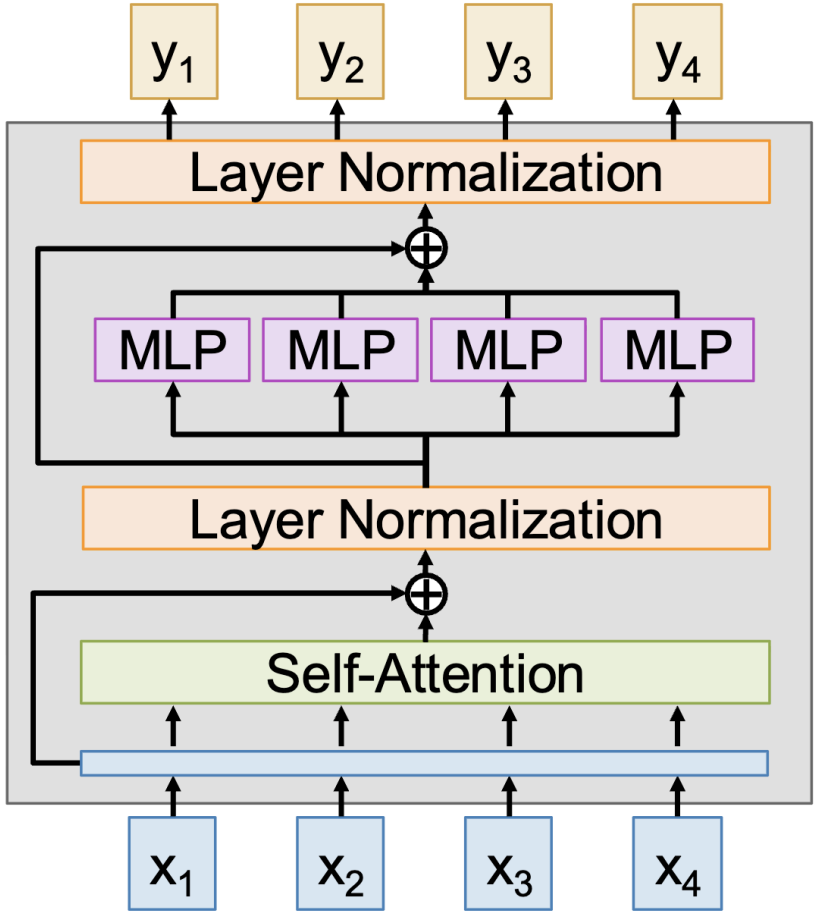


The Transformer

Transformer Block

- Input: Set of vectors x
- Output: Set of vectors y
- Self-Attention is the only interaction between vectors
- LayerNorm and MLP work on each vector independently
- Highly scalable and parallelizable, most of the compute is just 6 matmuls:

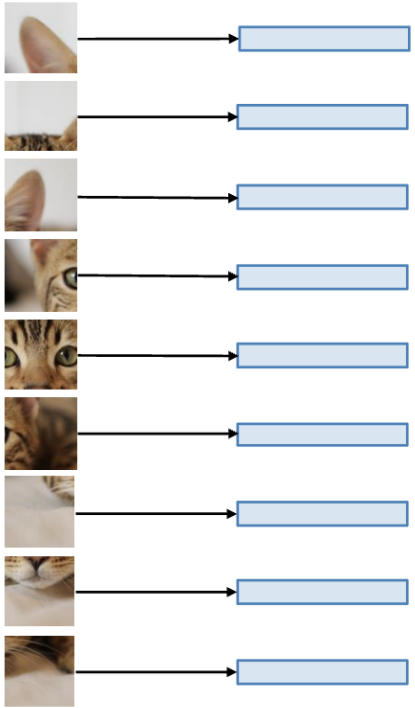
4 from Self-Attention 2 from MLP



Vision Transformer

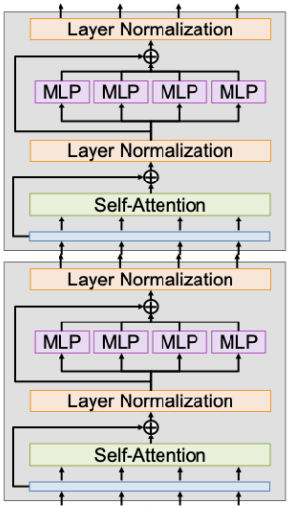
+ Average pool NxD vectors to1xD, apply a linear layerD=>C to predict class scores

Input image:
e.g. 224x224x3



Break into patches
e.g. 16x16x3

Flatten and apply a linear transform 768 => D



D-dim vector per patch are the input vectors to the Transformer

Don't use any masking; each image patch can look at all other image patches

Use positional encoding to tell the transformer the 2D position of each patch

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Mixture of Experts (MoE)

Learn E separate sets of MLP weights ineach block; each MLP is an expert

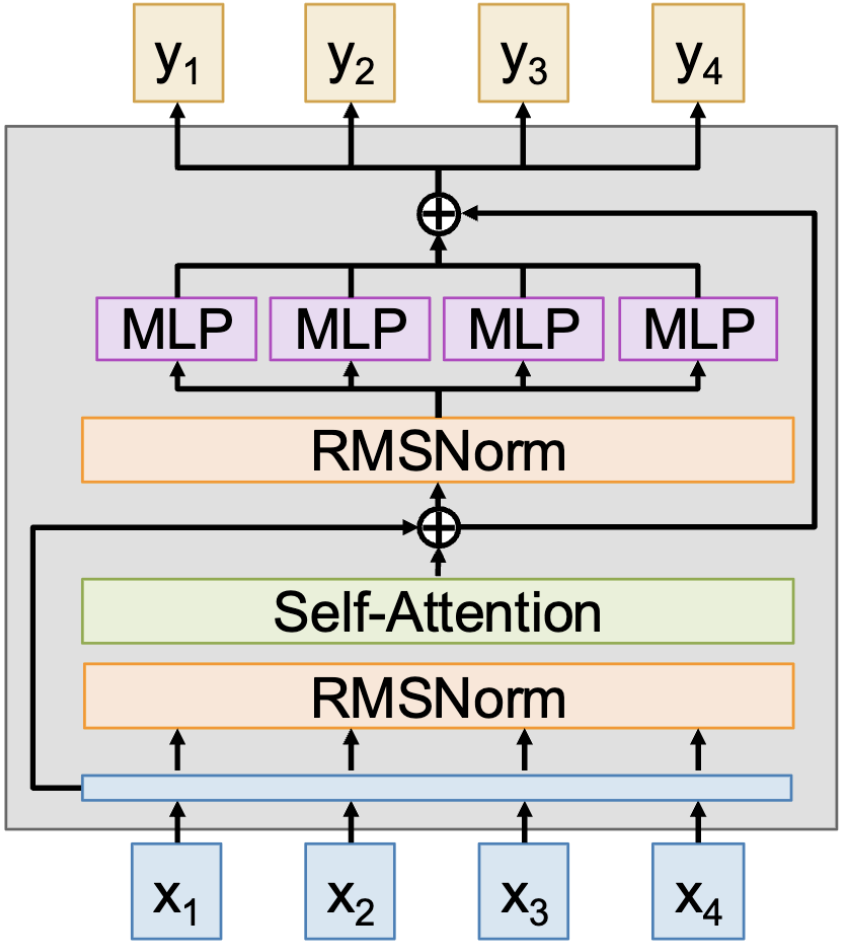
W1: [D x 4D] => [E x D x 4D]

W2: [4D x D] => [E x 4D x D]

Each token gets routed to A < E of the experts. These are the active experts.

Increases params by E, But only increases compute by A

All of the biggest LLMs today (e.g.GPT4o, GPT4.5, Claude 3.7, Gemini 2.5Pro, etc) almost certainly use MoE andhave > 1T params; but they don't publishdetails anymore



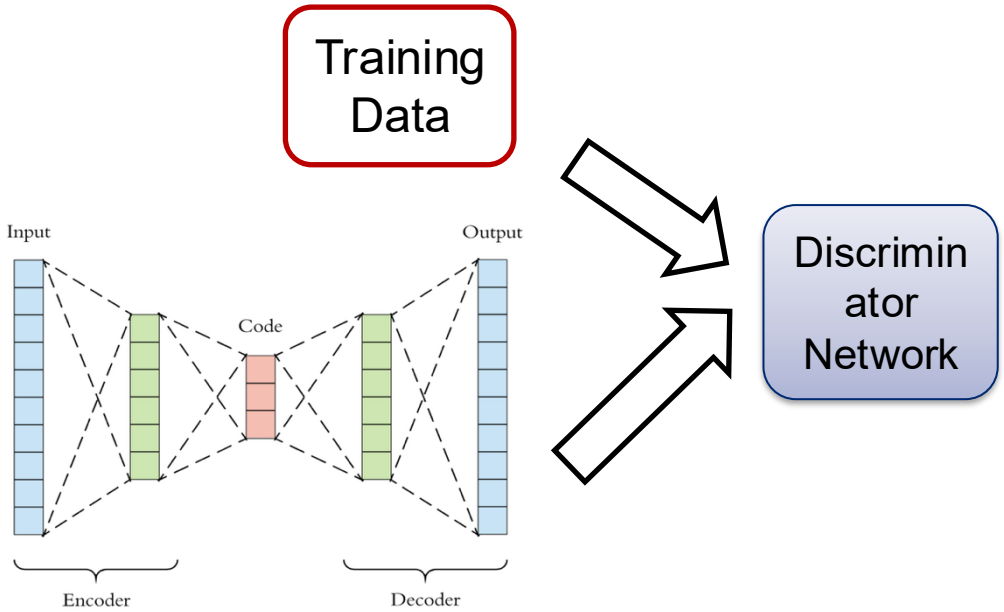
Generative Model: Generative Adversarial Networks

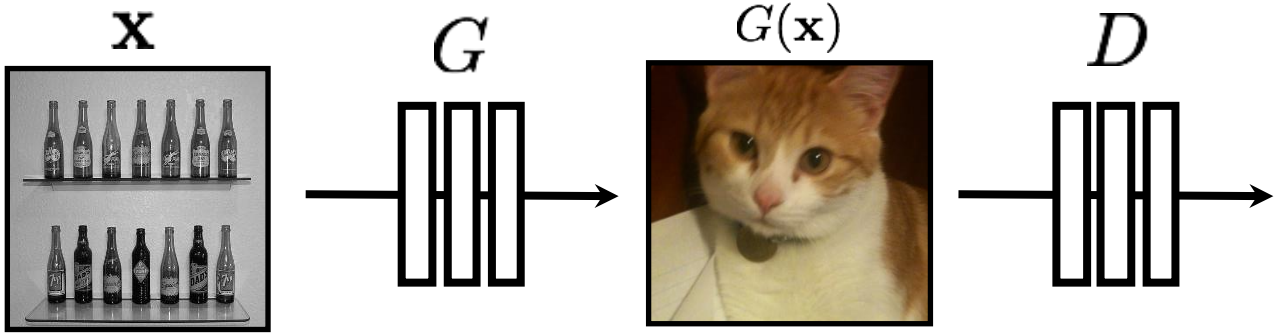
Generator network has similar structure to the decoder of our autoencoder

- Maps from some latent space to images

We train it in an adversarial manner against a discriminator network

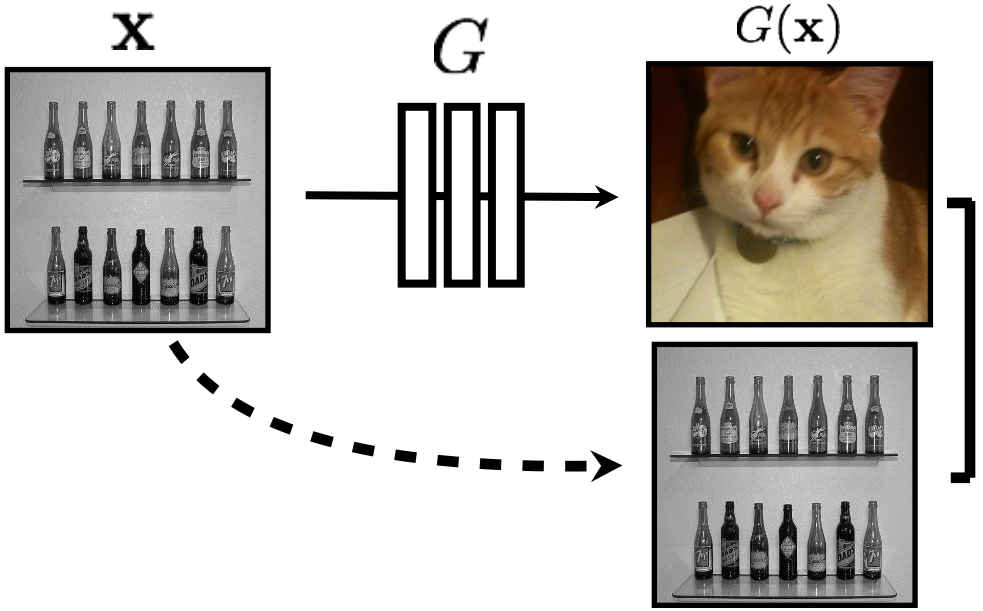
- Generator takes image noise, and tries to create output indistinguishable from training data
- Discriminator tries to distinguish between generator output and training data





$$\arg\min_G \max_D \; \mathbb{E}_{\mathbf{x},\mathbf{y}}[\; \log D(G(\mathbf{x})) \quad + \quad \log(1 - D(\mathbf{y})) \;\;]$$

[Goodfellow et al., 2014]



$$\arg\min_G \max_D \; \mathbb{E}_{\mathbf{x},\mathbf{y}}[\; \log D(G(\mathbf{x})) \;+\; \log(1 - D(\mathbf{y}))\;]$$

[Goodfellow et al., 2014]
[Isola et al., 2017]

More Examples of Image-to-Image Translation with GANs

We have pairs of corresponding training images

Conditioned on one of the images, sample from the distribution of likely corresponding images

Segmentation to Street Image

input output

Aerial Photo To Map

input output

Edges to Image

Limitations

The unconditional models above must be trained per-category:

- We have a separate model for every category – an animal face model, broccoli model, horse model, etc…

What if we want to generate an image from **any** description?

-> diffusion and text-to-image models

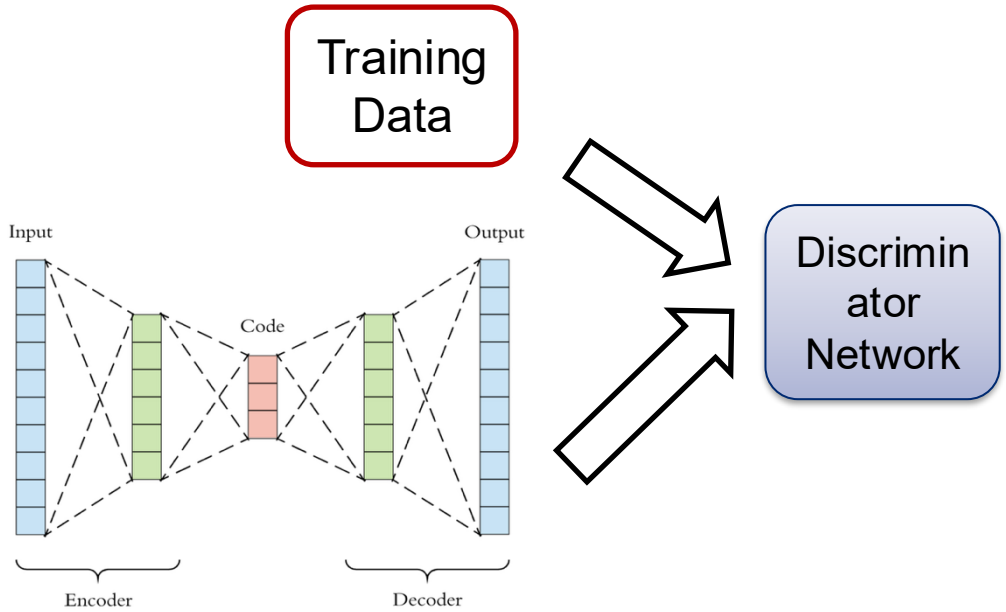
Generative Model: Generative Adversarial Networks

Generator network has similar structure to the decoder of our autoencoder

- Maps from some latent space to images

We train it in an adversarial manner against a discriminator network

- Generator takes image noise, and tries to create output indistinguishable from training data
- Discriminator tries to distinguish between generator output and training data



Recall: The Space of All Images

Lets consider the space of all 100x100 images

Now lets randomly sample that space…

Conclusion: Most images are noise

Question:
What do we expect a random uniform sample of all images to look like?

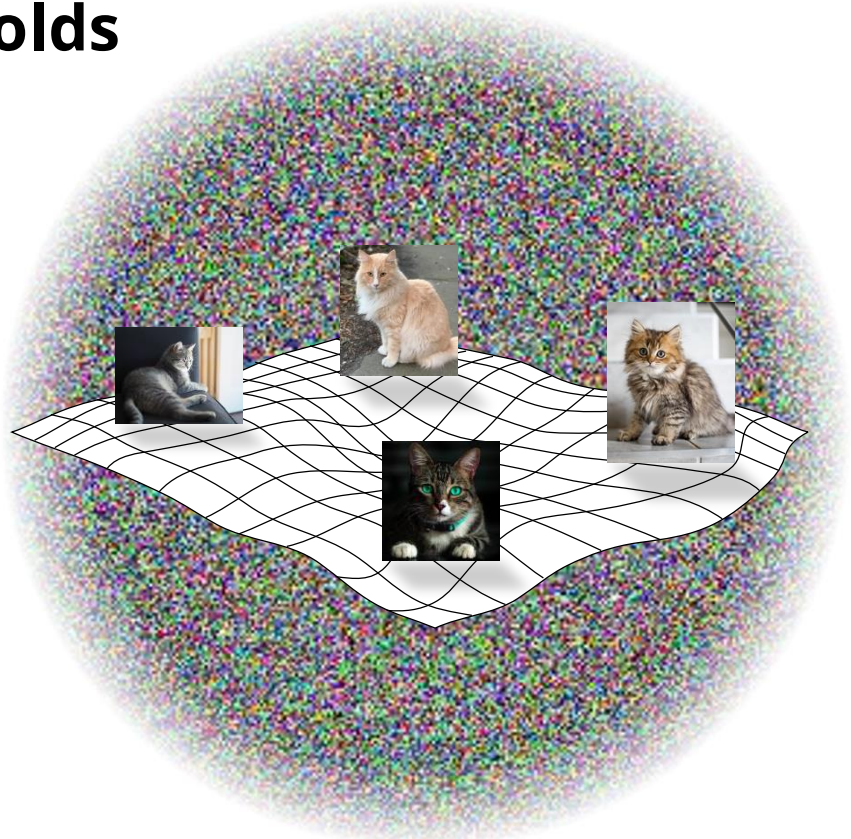
```
pixels = np.random.rand(100,100,3)
```

Recall: Natural Image Manifolds

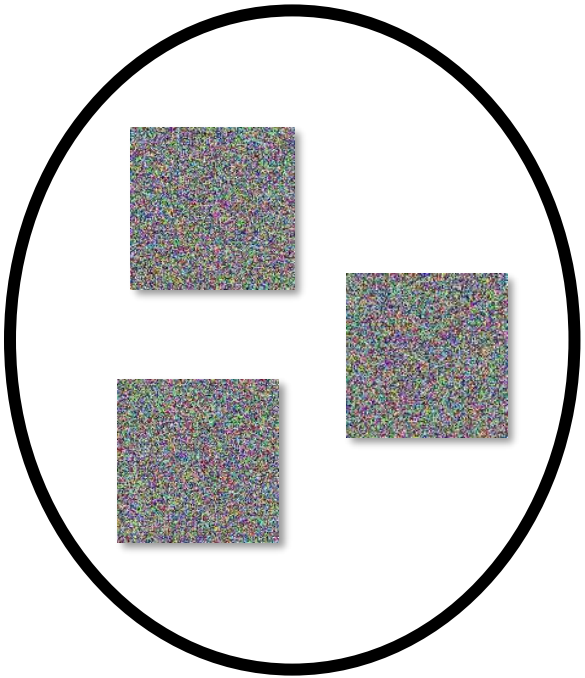
Most images are “noise”

“Meaningful” images tend to form some manifold within the space of all images

Images of a particular class fall on manifolds within that manifold…

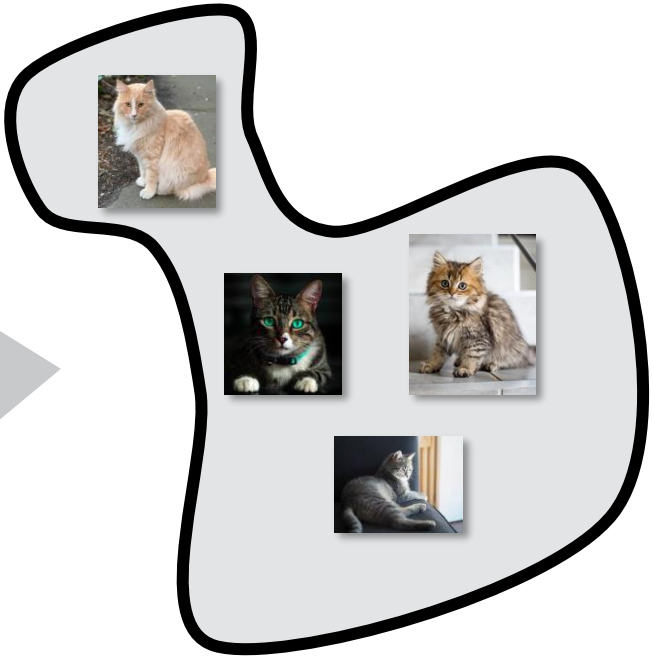


The Space of All Images



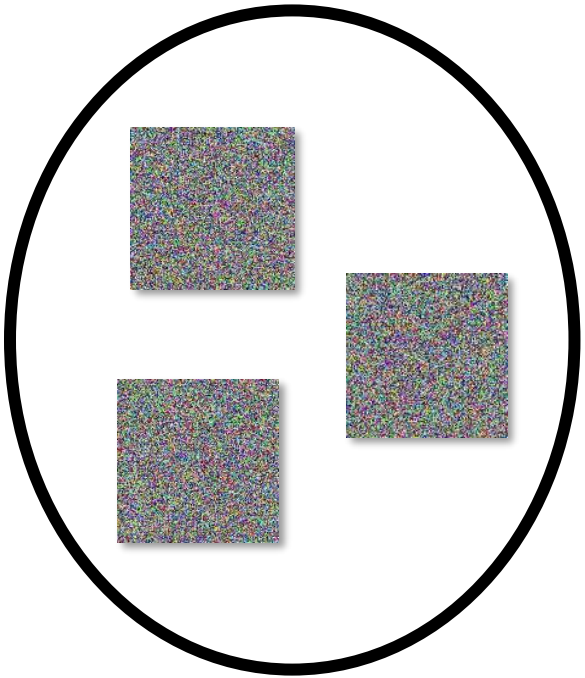
Random images

Diffusion



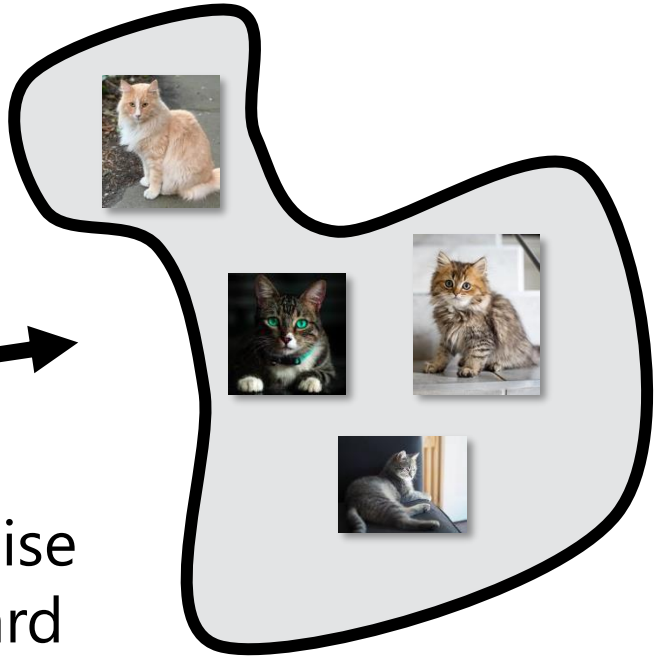
Manifold of cat images

Slide concept: Steve Seitz



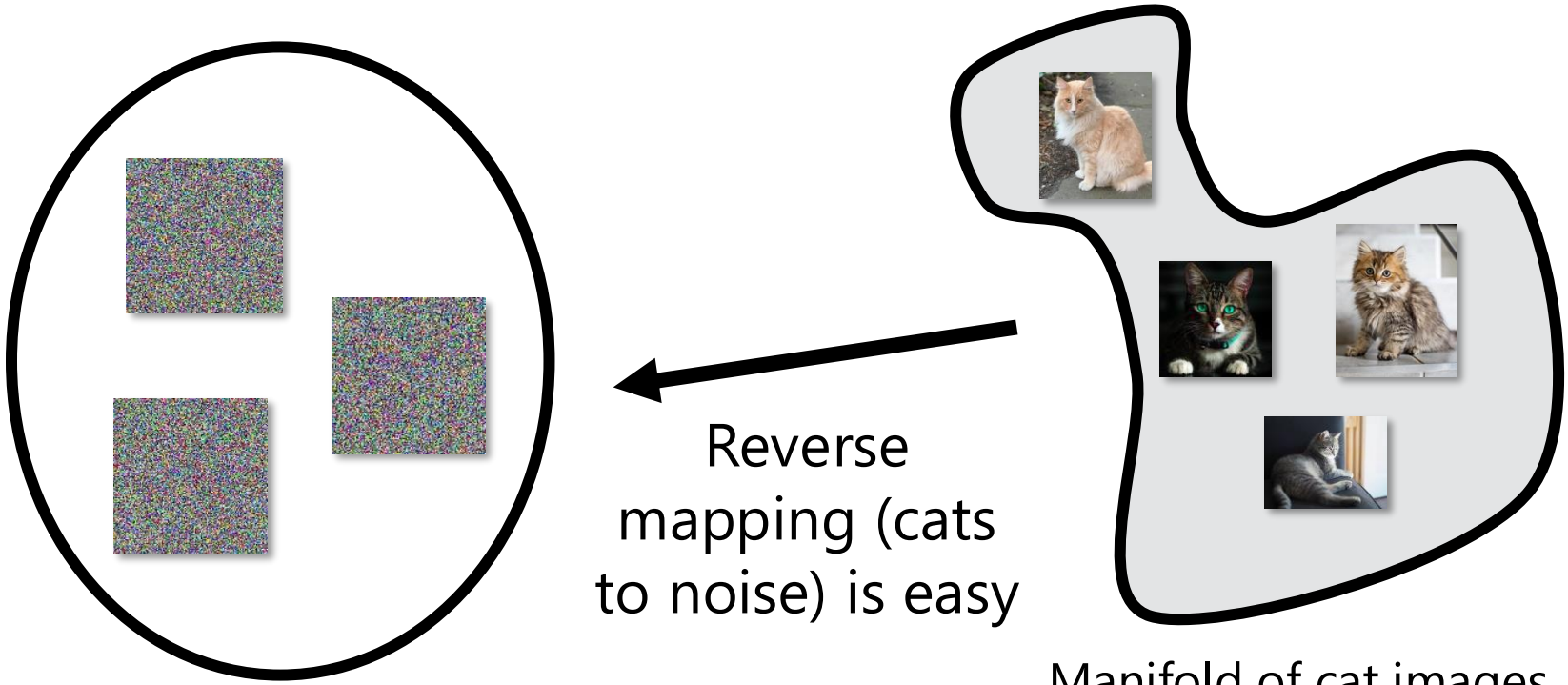
Random images

Forward mapping (noise to cats) is hard

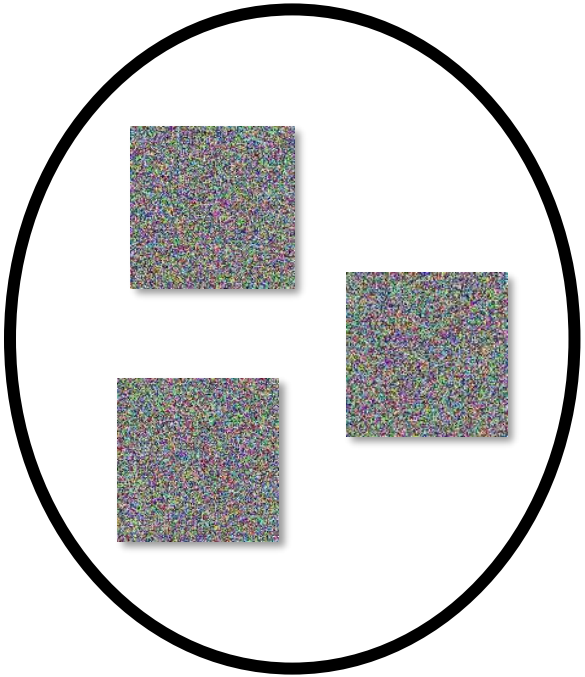


Manifold of cat images

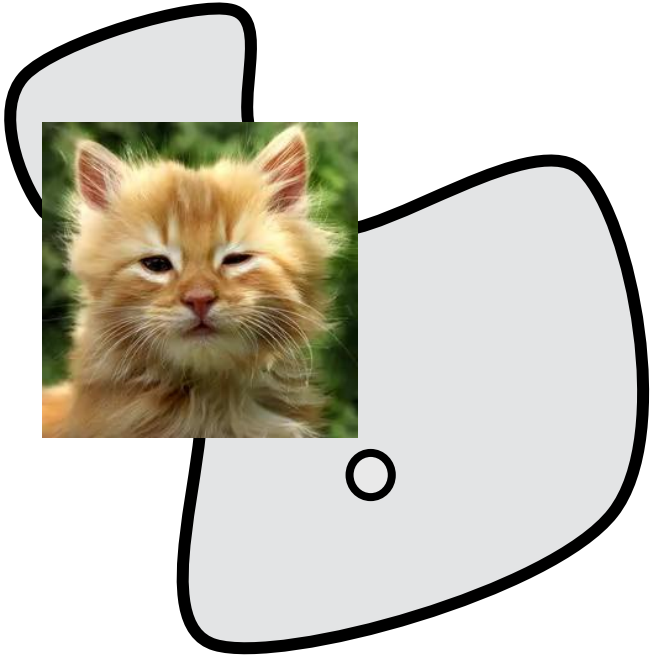
Slide concept: Steve Seitz



Slide concept: Steve Seitz

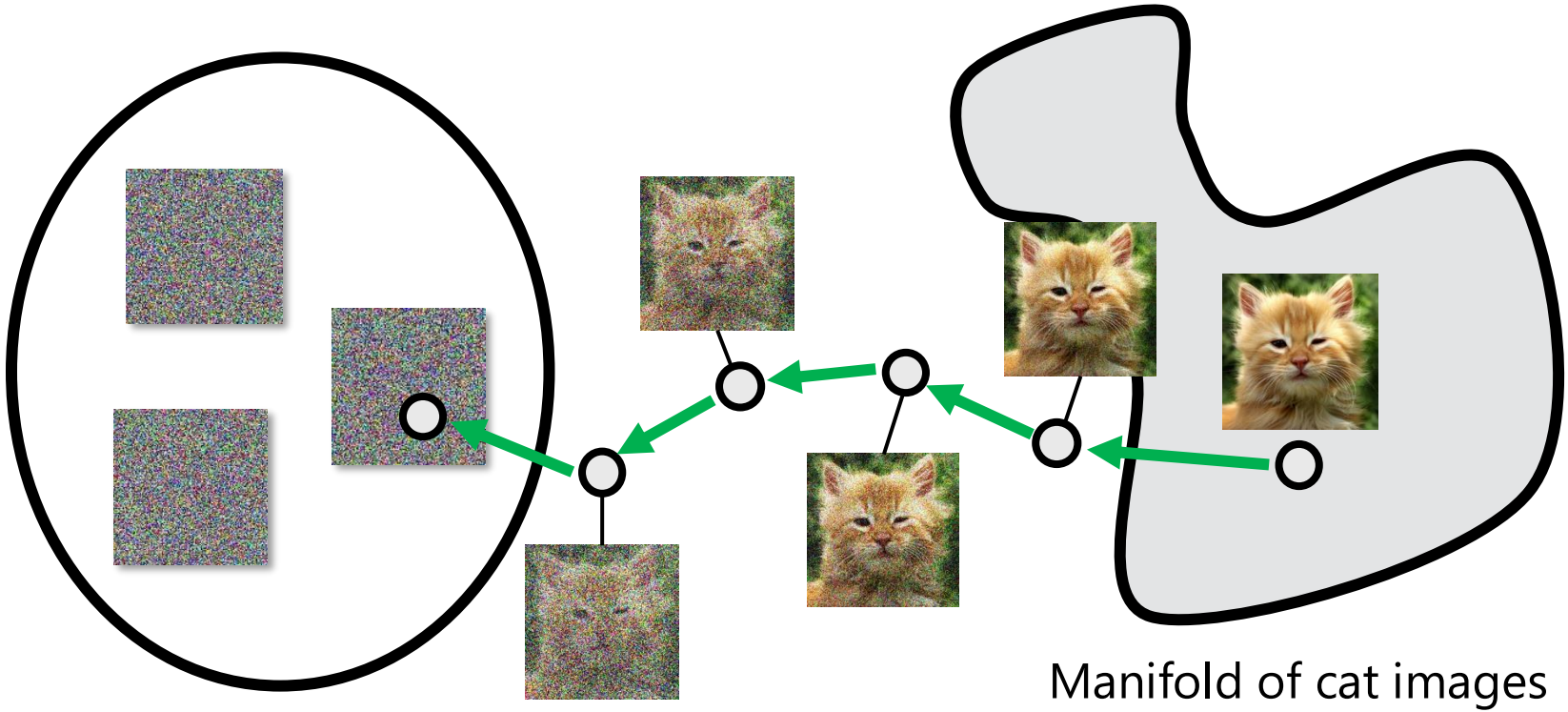


Random images

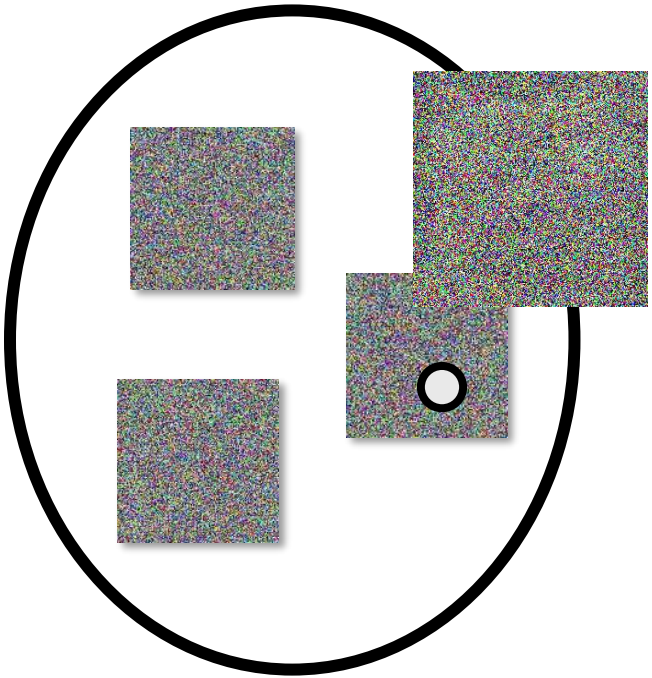


Manifold of cat images

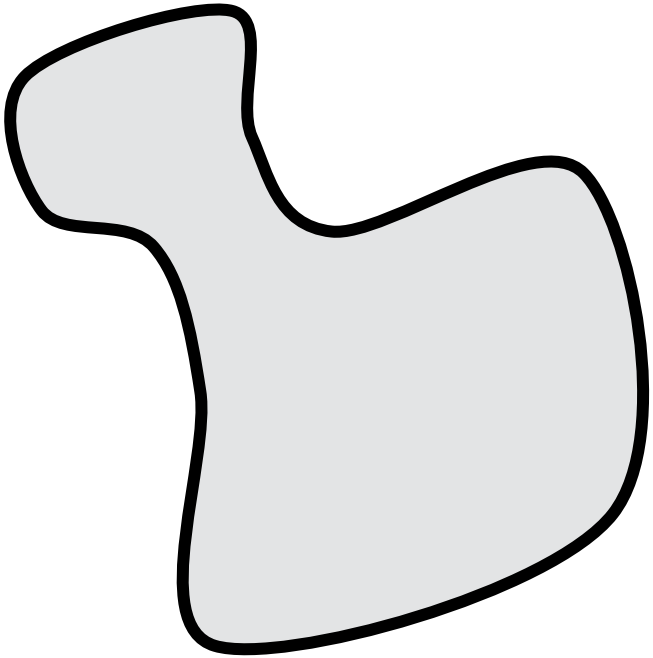
Slide concept: Steve Seitz



Slide concept: Steve Seitz

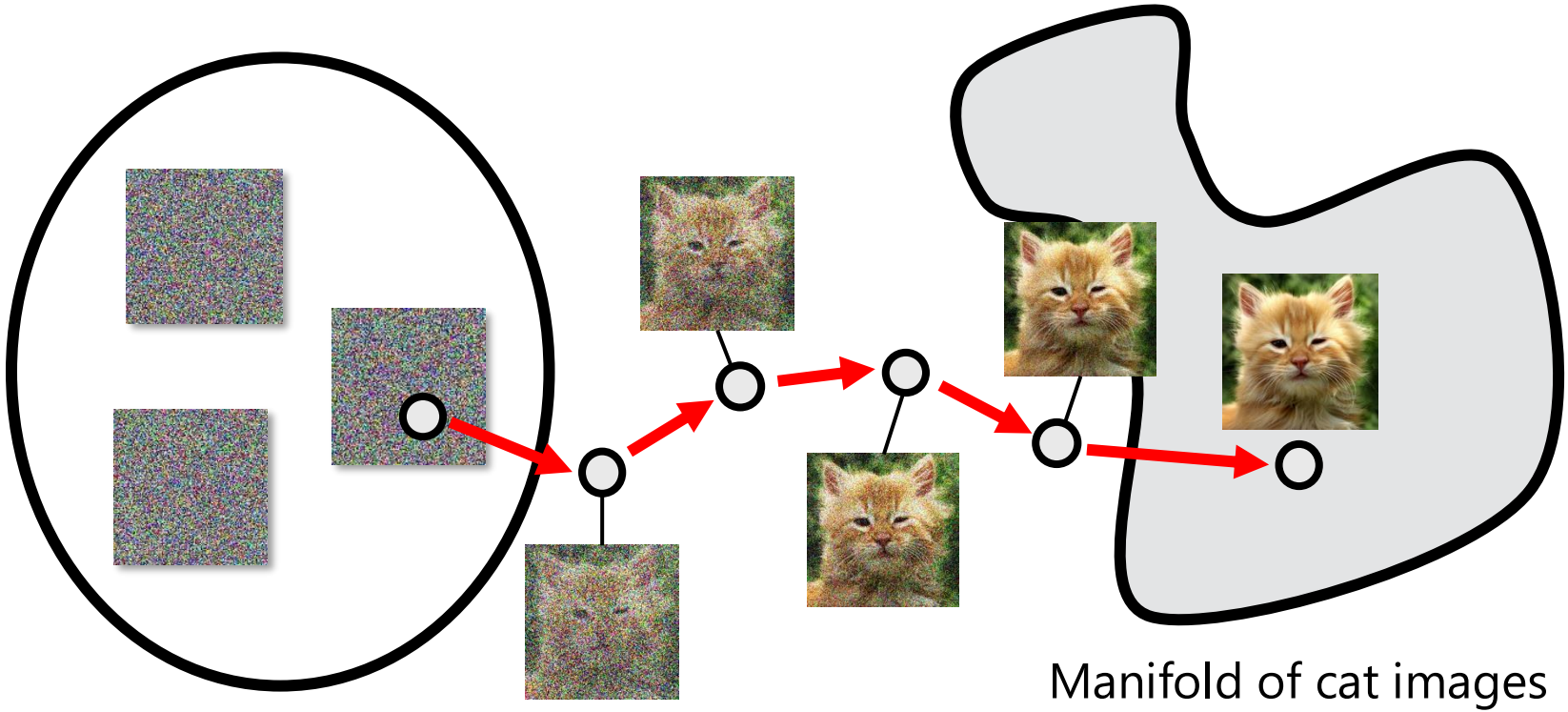


Random images

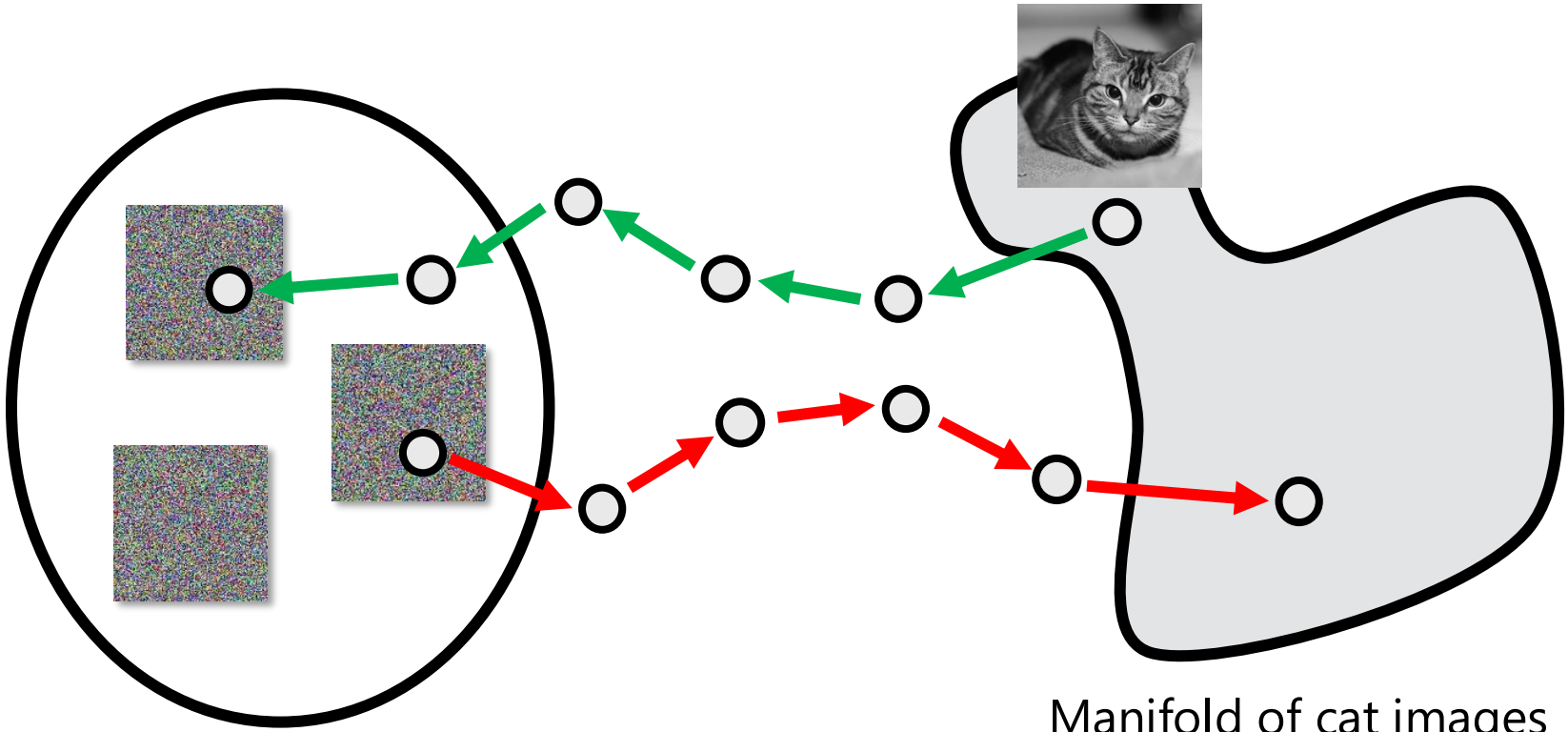


Manifold of cat images

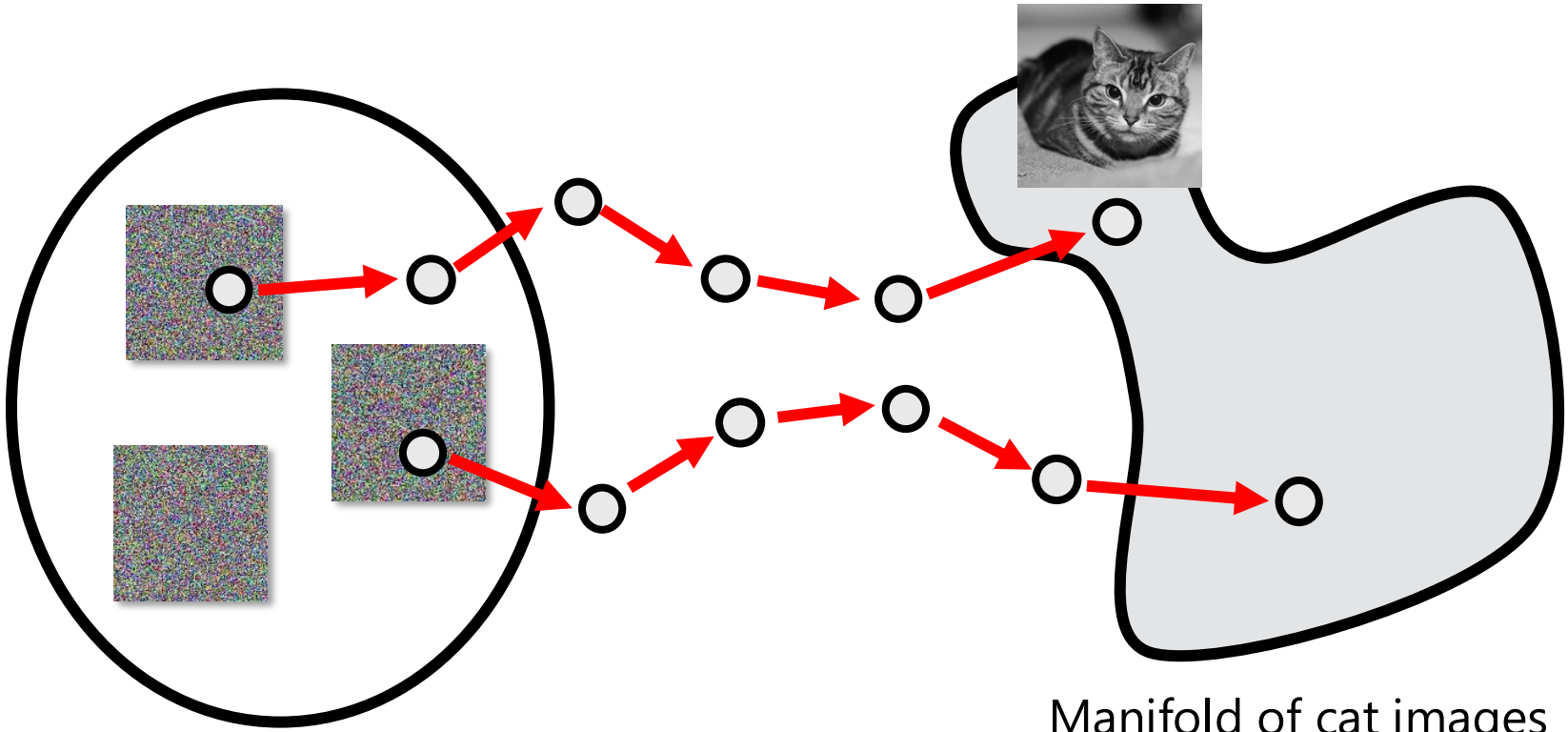
Slide concept: Steve Seitz



Slide concept: Steve Seitz



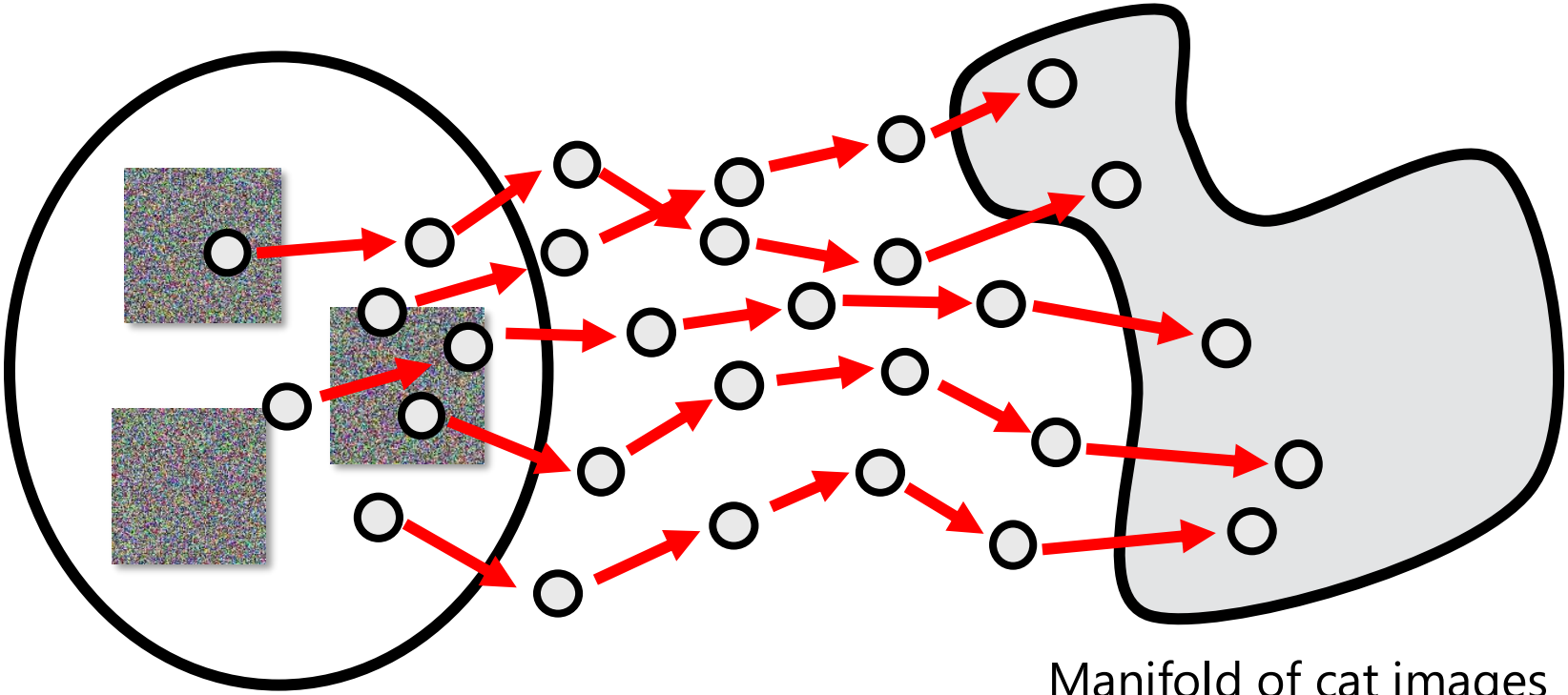
Slide concept: Steve Seitz



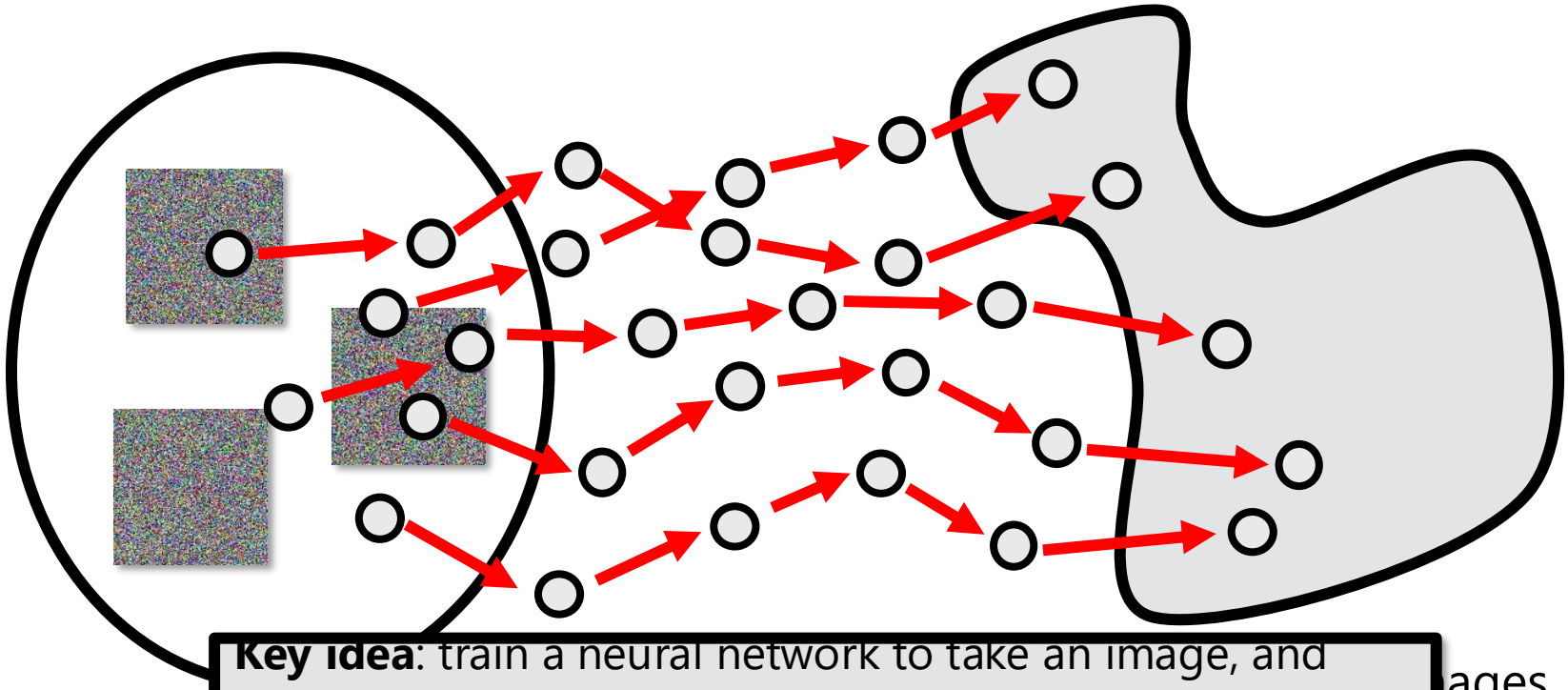
Random images

Manifold of cat images

Slide concept: Steve Seitz



Slide concept: Steve Seitz



Rand ages

Key idea: train a neural network to take an image, and predict the corresponding arrow above; that is, predict to convert a noisy image to a slightly less noisy image that is closer to the desired image manifold, using the examples above to train.

Slide concept: Steve Seitz

Denoising diffusion neural network

Diffusion neural network

This network can be a U-Net or other suitable image-to-image network

Imagen

"Sprouts in the shape of text 'Imagen' coming out of a fairytale book."

"A dragon fruit wearing karate belt in the snow."